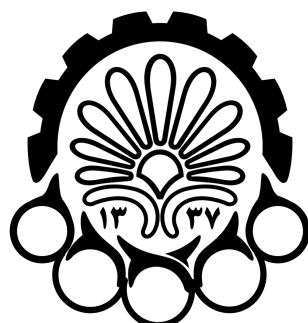




دانشگاه صنعتی امیر کبیر
(پلی تکنیک تهران)

دانشکده مهندسی کامپیوتر

پایان نامه پروژه کارشناسی

عنوان پروژه:

ترکیب تحلیل ساختاری کد با استدلال مدل های زبانی بزرگ و معماری چندعامله
برای کاهش هشدارهای کاذب در تحلیل آسیب پذیری کد

نگارنده:

علی ناصری نیا

استاد راهنما:

حمیدرضا شهریاری کاهکشی

تابستان ۱۴۰۵

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

اینجانب علی ناصری‌نیا متعهد می‌شوم که مطالب مندرج در این پایان‌نامه حاصل کار پژوهشی اینجانب تحت نظارت و راهنمایی اساتید دانشگاه صنعتی امیرکبیر بوده و به دستاوردهای دیگران که در این پژوهش از آن‌ها استفاده شده است مطابق مقررات و روال متعارف ارجاع و در فهرست منابع و مآخذ ذکر گردیده است. این پایان‌نامه قبلاً برای احراز هیچ مدرک هم‌سطح یا بالاتر ارائه نگردیده است.

در صورت اثبات تخلف در هر زمان، مدرک تحصیلی صادر شده توسط دانشگاه از درجه اعتبار ساقط بوده و دانشگاه حق پیگیری قانونی خواهد داشت.

کلیه نتایج و حقوق حاصل از این پایان‌نامه متعلق به دانشگاه صنعتی امیرکبیر است. هرگونه استفاده از نتایج علمی و عملی، واگذاری اطلاعات به دیگران یا چاپ و تکثیر، نسخه‌برداری، ترجمه و اقتباس از این پایان‌نامه بدون موافقت کتبی دانشگاه صنعتی امیرکبیر ممنوع است.

نقل مطالب با ذکر مآخذ بلامانع است.

علی ناصری‌نیا

امضا



سپاس‌گزاری

بدینوسیله مراتب قدردانی و امتنان خود را خدمت جناب آقای دکتر شهریاری‌کاهکشی، استاد راهنمای گرانقدر، که راهنمایی‌های خود را در تمام مراحل بر من ارزانی داشتند و بدون کمک‌های ایشان انجام این پروژه امکان‌پذیر نبود، ابراز و از زحمات ایشان بسیار سپاس‌گزار بوده و آرزومند توفیق روزافزون برایشان می‌باشم.

تابستان ۱۴۰۵

تقدیم

تقدیم به پدر و مادر عزیزم، که با مهربانی، صبر و حمایت بی‌دریغ خود همواره پشتوانه و چراغ راه من بوده‌اند. محبت و انگیزه‌ای که از شما دریافت کرده‌ام، همواره مشوق من در مسیر تحصیل و تلاش علمی بوده و بدون همراهی و دلگرمی‌های شما، دستیابی به این مرحله امکان‌پذیر نبوده است.

تابستان ۱۴۰۵

چکیده

تحلیل ایستا رویکردی رایج برای یافتن آسیب‌پذیری‌های نرم‌افزاری است، اما نرخ بالای هشدارهای نادرست همواره مانع اصلی پذیرش عملی آن بوده است. در این پژوهش، سامانه codey-security برای ترکیب تحلیل ساختاری کد با استدلال مدل‌های زبانی بزرگ در یک معماری چندعامله (جستجوگر و ارزیاب) طراحی و پیاده‌سازی شده است. این سامانه یافته‌های خام ابزارهای تحلیل ایستا را منحصراً بر پایه یک بسته شواهد ساخت‌یافته داوری می‌کند تا خطاهای سیستم را کاهش دهد.

ارزیابی‌های تجربی نشان می‌دهد بازبینی مبتنی بر شواهد توانسته است نرخ هشدار روی فایل‌های سالم را حدود ۸۶ درصد کاهش داده (از ۰/۰۵۰ به ۰/۰۰۷) و هم‌زمان دقت یافته‌ها را از ۰/۳۵۵ به ۰/۶۰۰ ارتقا دهد. اگرچه این پالایش سخت‌گیرانه منجر به افت یادآوری (از ۰/۰۳۷ به ۰/۰۱۰) گردید، اما بررسی‌ها نشان می‌دهد عامل زبانی در ۴۲ درصد موارد به دلیل کمبود شواهد کافی، هوشمندانه وضعیت «نامطمئن» را برگردانده است که مانع از تأییدهای نادرست می‌شود. در مجموع، این معماری نقطه کاری متعادلی برای سازمان‌های درگیر با حجم بالای هشدارهای کاذب فراهم می‌کند.

واژه‌های کلیدی:

تحلیل ایستا، مدل‌های زبانی بزرگ، معماری چندعامله، هشدار نادرست، استدلال مبتنی بر شواهد

فهرست مطالب

فصل اول - مقدمه	۱
۱.۱ بیان مسئله	۲
۱.۲ اهمیت و ضرورت پژوهش	۳
۱.۳ اهداف پژوهش	۵
۱.۴ دستاوردهای پژوهش	۶
۱.۵ روش پیشنهادی	۷
۱.۶ ساختار پایان نامه	۷
فصل دوم - مبانی نظری و کارهای پیشین	۸
۲.۱ تحلیل ایستا و مفهوم هشدار نادرست	۹
۲.۲ ابزارهای تحلیل ایستای به کاررفته در این پژوهش	۹
۲.۳ تحلیل ایستا-پویا و رویکردهای ترکیبی	۱۰
۲.۴ مدل های زبانی بزرگ و عامل های نرم افزاری	۱۱
۲.۵ استدلال مبتنی بر شواهد	۱۱
۲.۶ مجموعه داده جولیت	۱۱
۲.۷ کارهای پیشین	۱۳
۲.۸ جایگاه کار پیشنهادی	۱۴
۲.۹ چالش های ارزیابی سامانه های مبتنی بر مدل زبانی	۱۴
فصل سوم - طراحی و پیاده سازی سامانه	۱۶
۳.۱ معماری کلی	۱۷
۳.۲ فاز یک: تحلیل ایستا	۱۸
۳.۳ همبسته سازی یافته ها	۱۸
۳.۴ فاز دو: داوری چندعامله	۱۹
۳.۵ بسته شواهد	۱۹
۳.۶ پاک سازی کد و جلوگیری از نشت برچسب	۲۱
۳.۷ سیاست تطبیق یافته ها و معیارها	۲۲
۳.۸ شمای گزارش و نمونه های واقعی	۲۳

۳.۹	زیرساخت اجرا و تکرارپذیری	۲۵
۳.۱۰	مدل زبانی، ارائه‌دهنده و پیکربندی اجرا	۲۶

فصل چهارم - ارزیابی تجربی ۲۹

۴.۱	پرسش‌های پژوهش	۳۰
۴.۲	مجموعه داده و آماده‌سازی آن	۳۰
۴.۳	طراحی آزمایش‌ها	۳۱
۴.۴	معیارها و سیاست تطبیق	۳۲
۴.۵	نتایج اصلی	۳۲
۴.۶	تحلیل کیف تصمیم	۳۵
۴.۷	نتایج به تفکیک نوع آسیب‌پذیری	۳۶
۴.۸	بررسی در مقیاس کامل	۳۶
۴.۹	ارزیابی موتور شواهد در پایتون	۳۷
۴.۱۰	ارزیابی موتور شواهد در جولیت و دیویگن	۳۸
۴.۱۱	ساختار هشدارهای نادرست و تحلیل خطاها	۴۰
۴.۱۲	بازه‌های اطمینان و حساسیت به اندازه نمونه	۴۲
۴.۱۳	هزینه، زمان و پایداری اجرا	۴۳
۴.۱۴	تفکیک زبانی نتایج	۴۳
۴.۱۵	تحلیل کیفی یک تصمیم و ارزش امتیاز اعتماد	۴۴
۴.۱۶	تهدیدهای روایی	۴۵

فصل پنجم - نتیجه‌گیری و کارهای آینده ۴۶

۵.۱	جمع‌بندی	۴۷
۵.۲	محدودیت‌ها	۴۹
۵.۳	کارهای آینده	۴۹

فهرست مراجع ۵۱

فهرست شکل‌ها

- شکل ۱.۱ - نمونه‌ای از کد آسیب‌پذیر از نوع سرریز عدد صحیح ۴
- شکل ۱.۲ - نسخه اصلاح‌شده همان سناریو با بررسی پیش از محاسبه ۴
- شکل ۱.۳ - نمای کلی سامانه پیشنهادی و جایگاه داوری چندعامله مبتنی بر شواهد ۳
- شکل ۲.۱ - نسخه آسیب‌پذیر یک سناریوی جولیت ۱۲
- شکل ۲.۲ - نسخه سالم همان سناریو؛ تنها تفاوت، یک شرط کنترلی است ۱۲
- شکل ۲.۳ - پنج محور چالش‌برانگیز ارزیابی سامانه‌های مبتنی بر مدل زبانی ۱۵
- شکل ۳.۱ - معماری سه‌فاز سامانه و جریان داده میان فازها ۱۷
- شکل ۳.۲ - نمونه ساده‌شده بسته شواهد جستجوگر ۲۰
- شکل ۳.۳ - قطعه کد پس از پاک‌سازی؛ حاشیه‌نویسی‌ها محو و شناسه‌های افشاگر بازنام‌گذاری شده‌اند ۲۱
- شکل ۳.۴ - زنجیره ادغام یافته‌های تکراری و تشکیل گروه‌های بازبینی ۱۸
- شکل ۴.۱ - مقایسه دقت، یادآوری و نرخ هشدار روی فایل سالم در چهار پیکربندی ۳۳
- شکل ۴.۲ - ماتریس سردرگمی چهار پیکربندی در دانه‌بندی یافته و دانه‌بندی فایل ۳۴
- شکل ۴.۴ - اثر شواهد ساختاری بر تصمیم‌های عامل زبانی ۳۵
- شکل ۴.۵ - توان موتور شواهد در برابر ابزار پایه روی هر سه مجموعه ۳۹

فهرست جدول‌ها

- جدول ۲.۱ - ابزارهای تحلیل ایستای ارزیابی‌شده برای فاز یک ۱۰
- جدول ۲.۲ - جایگاه کار پیشنهادی در میان کارهای پیشین ۱۳
- جدول ۳.۱ - سه خروجی ممکن عامل ارزیاب ۱۹
- جدول ۳.۲ - سیاست تطبیق یافته‌ها با برجسب واقعی ۲۲
- جدول ۳.۳ - شمای گزارش نهایی سامانه ۲۳
- جدول ۳.۴ - دامنه پیشنهاد و نمونه‌های واقعی به‌کاررفته در گزارش ۲۴
- جدول ۳.۵ - پیکربندی مدل زبانی و اتصال سرویس داوری ۲۶
- جدول ۴.۱ - مشخصات مجموعه داده ارزیابی و پیکربندی آزمایش ۳۰
- جدول ۴.۲ - نتایج اصلی چهار آزمایش روی ۶۰۰ نمونه متعادل ۳۲
- جدول ۴.۳ - قیف تصمیم عامل زبانی در هر پیکربندی ۳۵
- جدول ۴.۴ - نتایج به تفکیک نوع آسیب‌پذیری ۳۶
- جدول ۴.۵ - نتایج موتور قطعی روی محک ۴۸ نمونه‌ای پایتون ۳۷
- جدول ۴.۶ - نتایج موتور قطعی روی جولیت و دیویگن ۳۸
- جدول ۴.۷ - تفکیک توان موتور شواهد بر پایه خانواده آسیب‌پذیری، زبان و دسته نقطه خطر ۴۰
- جدول ۴.۸ - ساختار هشدارهای نادرست پیکربندی A بر پایه ابزار و خانواده گزارش‌شده ۴۰
- جدول ۴.۹ - برآورد نقطه‌ای و بازه اطمینان ویلسون ۹۵ درصدی معیارهای اصلی ۴۲
- جدول ۴.۱۰ - نتایج چهار پیکربندی به تفکیک زبان ۴۳
- جدول ۵.۱ - پاسخ پنج پرسش پژوهش همراه با شاهد تجربی ۴۷

فصل اول

مقدمه

۱.۱ بیان مسئله

تحلیل ایستا^۱ یکی از پرکاربردترین رویکردها برای یافتن آسیب‌پذیری‌های امنیتی در نرم‌افزار است. در این رویکرد، کد پروژه بدون اجرا شدن و با استفاده از تحلیل نحوی، کنترل جریان داده و مجموعه‌ای از قواعد از پیش تعریف‌شده بررسی می‌شود و ابزار، محل‌هایی از کد را که ممکن است دربردارنده خطا باشند به‌عنوان هشدار گزارش می‌کند. مزیت اصلی این رویکرد آن است که نیازی به اجرای برنامه یا آماده‌سازی ورودی‌های خاص نبوده و در نتیجه می‌توان آن را در مراحل ابتدایی چرخه توسعه و بر روی کل مخزن نرم‌افزاری به کار برد. به همین دلیل، ابزارهایی مانند [۱] cppcheck و [۲] flawfinder در فرآیندهای یکپارچه ساخت و زنجیره‌های ابزار توسعه جایگاه تثبیت‌شده‌ای پیدا کرده‌اند.

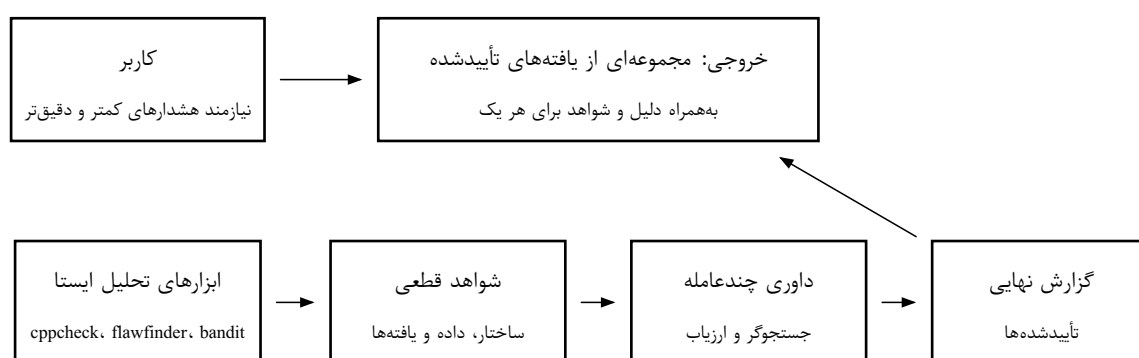
با این حال، مهم‌ترین چالش عملی ابزارهای تحلیل ایستا، نرخ بالای هشدارهای نادرست^۲ است. هشدار نادرست یعنی ابزار محلی را به‌عنوان محل آسیب‌پذیری معرفی کرده است، در حالی که آن مسیر در واقع اجرا نمی‌شود، یا شرط لازم برای بهره‌برداری مهاجم هرگز برقرار نمی‌شود. بررسی‌های انجام‌شده روی پروژه‌های واقعی نشان می‌دهد که سهم قابل‌توجهی از هشدارهای این ابزارها در عمل نادرست است و همین موضوع سبب می‌شود توسعه‌دهندگان به‌تدریج به هشدارها بی‌اعتماد شوند و در نهایت ابزار را کنار بگذارند. بنابراین مسئله اصلی، افزایش تعداد هشدارها نیست، بلکه افزایش نسبت هشدارهای مفید و کاهش هشدارهای نادرست در عین حفظ توانایی کشف آسیب‌پذیری‌های واقعی است. در سال‌های اخیر، مدل‌های زبانی بزرگ^۳ توانسته‌اند درک معنایی از کد به دست آورند و این امکان را ایجاد کرده‌اند که بخشی از قضاوت‌های نیازمند فهم زمینه اجرای برنامه، به‌جای قواعد صرفاً نحوی، به این مدل‌ها واگذار شود. برای نمونه، بازبینی هشدار یک ابزار ایستا توسط یک مدل زبانی می‌تواند بررسی کند که آیا مسیر داده واقعاً قابل دسترسی است، آیا مقدار مورد استفاده پیش از رسیدن به نقطه خطر اعتبارسنجی می‌شود و آیا نام توابع فراخوانی‌شده با الگوهای شناخته‌شده خطرناک همخوانی دارد یا خیر. با این حال، اتکال صرف به مدل زبانی نیز ریسک‌هایی به همراه دارد. این مدل‌ها ممکن است یافته‌ای را بدون شواهد کافی تأیید کنند، دلایل نادرست تولید کنند و در صورت مواجهه با کد مبهم، به‌جای اعلام عدم قطعیت، پاسخ قطعی تولید کنند [۱۰] [۱۵]. به همین دلیل، استفاده از آن‌ها به‌عنوان جایگزین کامل ابزارهای تحلیل ایستا، دست‌کم در ارزیابی‌های کمی، نتیجه رضایت‌بخشی به همراه نداشته است.

^۱ Static Analysis

^۲ False Positive

^۳ Large Language Model

مسئله‌ای که این پژوهش بر آن تمرکز دارد، طراحی و ارزیابی یک سامانه ترکیبی با نام codey-security است که در آن تحلیل ایستا نقش پیشنهاددهنده نامزدها را بر عهده می‌گیرد و دو عامل مستقل جستجوگر^۴ و ارزیاب^۵ نقش داور مبتنی بر شواهد را ایفا می‌کنند. همان‌طور که در شکل ۱.۳ مشاهده می‌شود، هدف این سامانه آن است که هشدارهای نادرست ابزارهای ایستا را بدون از دست دادن آسیب‌پذیری‌های واقعی کاهش دهد. برای آنکه ادعای کاهش هشدار نادرست قابل دفاع باشد، لازم است ابتدا مشخص شود که معیار «کاهش» دقیقاً بر چه پایه‌ای سنجیده می‌شود. این موضوع در فصل چهارم با معرفی سیاست تطبیق یافته‌ها و تفکیک دانه‌بندی معیارها بررسی می‌شود، اما به‌طور خلاصه، در این پایان‌نامه دو دانه‌بندی متمایز گزارش می‌شود: دانه‌بندی یافته که تطابق میان هشدار ابزار و محل واقعی خطا را می‌سنجد، و دانه‌بندی فایل که میزان آلودگی هر فایل سالم به هشدار را اندازه می‌گیرد.



شکل ۱.۳: نمای کلی سامانه پیشنهادی و جایگاه داوری چندعامله مبتنی بر شواهد

۱.۲ اهمیت و ضرورت پژوهش

آسیب‌پذیری‌های ایمنی حافظه، از جمله سرریز بافر مبتنی بر هیپ^۶ و سرریز عدد صحیح^۷، همچنان از پرتکرارترین و خطرناک‌ترین دسته‌های آسیب‌پذیری در نرم‌افزارهای نوشته‌شده به زبان‌های C/C++ هستند. سازوکار این آسیب‌پذیری‌ها به‌گونه‌ای است که بسیاری از آن‌ها تنها با دنبال کردن جریان داده از منبع تا نقطه خطر و بررسی شرط‌های کنترل‌کننده آشکار می‌شوند (برای درک بهتر، تفاوت کد آسیب‌پذیر و نسخه اصلاح‌شده یک سرریز عدد صحیح در شکل‌های ۱.۱ و ۱.۲ نشان داده شده است). همین تحلیل جریان داده است که سبب می‌شود ابزارهای ایستا بیشترین حجم هشدارهای نادرست را در این دسته‌ها تولید کنند. این تنش، یعنی ضرورت پوشش آسیب‌پذیری در کنار پرهیز از هشدار نادرست، مسئله مرکزی این پژوهش است.

^۴ Scanner Agent

^۵ Verifier Agent

^۶ Heap-based Buffer Overflow

^۷ Integer Overflow

CWE190_Integer_Overflow__int_rand_add_73b_bad.cpp

```
void badSink(list<int> dataList)
{
    int data = dataList.back();
    {
        /* POTENTIAL FLAW: Adding 1 to data could
           cause an overflow */
        int result = data + 1;
        printIntLine(result);
    }
}
```

شکل ۱.۱: نمونه‌ای از کد آسیب‌پذیر از نوع سرریز عدد صحیح

CWE190_Integer_Overflow__int_rand_add_73b_good.cpp

```
void goodB2GSink(list<int> dataList)
{
    int data = dataList.back();
    if (data < INT_MAX) {
        int result = data + 1;
        printIntLine(result);
    } else {
        printLine("data value is too large ...");
    }
}
```

شکل ۱.۲: نسخه اصلاح‌شده همان سناریو با بررسی پیش از محاسبه

اهمیت موضوع زمانی بیشتر می‌شود که هزینه بررسی دستی یک هشدار نادرست در نظر گرفته شود. هر هشدار نادرست، زمان توسعه‌دهنده را صرف خواندن کدی می‌کند که در نهایت سالم ارزیابی می‌شود و همین هزینه، انگیزه استفاده از ابزار را از بین می‌برد. در مقابل، یک آسیب‌پذیری که دیده نشود، ممکن است ماه‌ها یا سال‌ها در سامانه باقی بماند. بنابراین سامانه‌ای که بتواند با هزینه محاسباتی اندک، سیگنال اولیه ابزارهای ایستا را پالایش کند و تنها موارد دارای شاهد کافی را به توسعه‌دهنده نشان دهد، ارزش عملی روشنی دارد.

۱.۳ اهداف پژوهش

هدف کلی این پژوهش، طراحی، پیاده‌سازی و ارزیابی کمی سامانه codey-security برای کاهش هشدارهای نادرست تحلیل ایستا در کد C و Python به کمک بازبینی مبتنی بر شواهد با دو عامل زبانی مستقل است. پشتیبانی نحوی از C++ برای پوشش کامل مجموعه جوییت حفظ شده است، اما دامنه زبانی اعلام‌شده پژوهش همان دو زبان ذکرشده در پروپوزال است. اهداف جزئی به شرح زیر بیان می‌شوند:

- طراحی معماری سه‌فاز شامل فاز شواهد قطعی، فاز داوری چندعامله جستجوگر و ارزیاب، و فاز ارزیابی مستقل، به‌گونه‌ای که مرز مسئولیت هر فاز روشن و قابل آزمون باشد.
- تعریف بسته شواهد ساخت‌یافته شامل قطعه کد پاک‌سازی‌شده، پیام ابزارها و شواهد ساختاری، به‌منظور آنکه تصمیم‌عامل‌های زبانی بر پایه شواهد و نه بر پایه حدس گرفته شود.
- پیاده‌سازی سازوکاری برای جلوگیری از نشت برچسب^۸ در مجموعه داده ارزیابی، به‌گونه‌ای که نتیجه آزمایش‌ها بازتاب توانایی واقعی سامانه باشد و نه توانایی خواندن پاسخ از درون فایل آزمون.
- تدوین چارچوب ارزیابی تکرارپذیر شامل سیاست تطبیق یافته‌ها، تفکیک دانه‌بندی معیارها و گزارش نتایج به تفکیک نوع آسیب‌پذیری.
- بررسی تجربی اثر هر جزء سامانه از طریق طراحی چهار آزمایش مقایسه‌ای و تحلیل نتایج آن‌ها.

دامنه آسیب‌پذیری این پژوهش همان دامنه تعریف‌شده در پروپوزال است: در زبان C سرریز بافر و سرریز عدد صحیح، و در زبان Python تزریق فرمان، تزریق کد، تزریق قالب سمت سرور، بازسازی ناامن داده و استفاده خطرناک از رابط‌های برنامه‌نویسی کاربردی بررسی می‌شود. پشتیبانی موتور شواهد از تزریق SQL و پیمایش مسیر، قابلیت کمکی تحلیل است و در ارزیابی اصلی ادعایی درباره آن‌ها مطرح نمی‌شود. فهرست دقیق شناسه‌های CWE و نمونه‌های واقعی متناظر، همراه با موارد اصلاح‌شده شناسه‌های نادرست اولیه، در بخش ۳.۸ آمده است.

^۸Label Leakage

۱.۴ دستاوردهای پژوهش

دستاوردهای اصلی این پژوهش به شرح زیر است:

- پیاده‌سازی یک سامانه سه‌فاز به‌صورت یک بسته نرم‌افزاری قابل اجرا، شامل اجرای ابزارهای تحلیل ایستا، استخراج ساختار نحوی، بازیابی مسیر منبع تا نقطه مصرف، پیشنهاد فرضیه توسط عامل جستجوگر و داوری آن توسط عامل ارزیاب.
- طراحی سازوکار سه‌گزینه‌ای برای داوری عامل زبانی (تأیید، رد، نامطمئن) که به سامانه اجازه می‌دهد در صورت مواجهه با کد مبهم یا کمبود شواهد، هوشمندانه اعلام عدم قطعیت کند و از تولید پاسخ‌های قطعی اما نادرست بپرهیزد.
- ارائه ماژول پاک‌سازی کد که با محوکردن حاشیه‌نویسی‌ها در جای خود و بازنام‌گذاری شناسه‌های افشاگر، برچسب واقعی را از دید مدل پنهان می‌کند و در عین حال شماره خط گزارش‌شده توسط ابزارها را تغییر نمی‌دهد.
- ارائه چارچوب ارزیابی تکرارپذیر شامل چهار آزمایش مقایسه‌ای، سیاست تطبیق یافته‌ها و شکستن امتیازهای برابر، معیارهای دانه‌بندی یافته و دانه‌بندی فایل و گزارش خودکار جدول مقایسه.
- به‌کارگیری مجموعه داده جولیت به‌عنوان مجموعه ارزیابی و گزارش نتایج آن بر روی مجموعه‌ای متعادل و لایه‌بندی‌شده از نظر نوع آسیب‌پذیری و برچسب.
- ارزیابی موتور شواهد قطعی روی محک مصنوعی جفتی پایتون و تابع‌های واقعی دیویگن، به‌همراه مقایسه رودرو با بندیت و فلافایندر.
- تحلیل نتایج و استخراج این نتیجه‌گیری که اثربخشی بازبینی مبتنی بر شواهد به کیفیت و دانه‌بندی شواهد ارائه‌شده وابسته است.
- ثبت فراداده کامل هر اجرا، شامل نام و نشانی مدل زبانی، نسخه ابزارها و محیط اجرا، به‌گونه‌ای که معیارها بتوانند بدون اجرای دوباره هیچ فراخوانی زبانی، از روی گزارش‌های بایگانی‌شده بازمحاسبه شوند.

برای آنکه این دستاوردها ملموس باشند، سه عدد محوری این پژوهش را می‌توان این‌گونه خلاصه کرد: بازبینی مبتنی بر شواهد، نرخ هشدار روی فایل سالم را از ۰/۰۵۰ به ۰/۰۰۷ رساند؛ عامل زبانی تنها با یادآوری ۰/۱۴۷، چهار برابر تحلیل ایستا آسیب‌پذیری یافت؛ و موتور قطعی بازیابی مسیر، بدون هیچ فراخوانی مدلی، روی محک پایتون به دقت ۰/۹۶ با نرخ هشدار ۰/۰۴۲ و یادآوری کامل رسید. هر سه عدد در فصل چهارم، همراه با بازه اطمینان، تفکیک خطاها و تحلیل هزینه گزارش شده‌اند؛ و همه آن‌ها از روی خروجی‌های بایگانی‌شده قابل بازتولیدند. همین شفافیت است که مرز ادعا را روشن می‌کند: کاهش هشدار نادرست و برتری یادآوری عامل زبانی، دو ادعایی‌اند که با داده موجود قابل دفاع‌اند، در حالی که رتبه‌بندی دقت پیکربندی‌های ترکیبی با این اندازه نمونه قابل اثبات نیست.

۱.۵ روش پیشنهادی

روش پیشنهادی این پژوهش، بر پایه این مشاهده استوار است که تحلیل ایستا و مدل زبانی، نقطه قوت مکمل دارند. ابزار ایستا نامزدهای بی شمار و ارزان تولید می کند، اما در قضاوت نهایی ضعیف است؛ مدل زبانی در فهم زمینه اجرا توانمند است، اما به تنهایی نامزد قابل اتکایی پیش نمی دهد. بر این اساس، در سامانه codey-security، ابزارهای ایستا تنها به عنوان تأمین کننده نامزد به کار می روند و تصمیم نهایی بر عهده دو عامل مستقل است: عامل جستجوگر فرضیه های آسیب پذیری را پیشنهاد می کند و عامل ارزیاب هر فرضیه را جداگانه، تنها بر پایه بسته شواهد ارائه شده، ارزیابی می کند. برای آنکه دآوری عامل های زبانی قابل اعتماد باشد، سه تصمیم طراحی اتخاذ شده است. نخست، شواهد به صورت ساخت یافته و با کمترین زمینه اضافی ارائه می شود تا هم حجم توکن کاهش یابد و هم تمرکز مدل بر همان نقطه مورد ادعا بماند. دوم، هر متنی که ممکن است برچسب را افشا کند، پیش از ارسال پاک سازی می شود. سوم، به جای دو گزینه ای بودن، سه خروجی تعریف شده است تا عامل بتواند در نبود شاهد کافی، صریحاً عدم قطعیت خود را اعلام کند.

۱.۶ ساختار پایان نامه

ساختار این پایان نامه به شرح زیر است. در فصل دوم، مبانی نظری شامل تحلیل ایستا، تحلیل ایستا-پویا، مدل های زبانی بزرگ و عامل های نرم افزاری، مجموعه داده جولیت و کارهای پیشین بررسی می شود و بخش پایانی آن به چالش های عملی ارزیابی سامانه های مبتنی بر مدل زبانی اختصاص دارد. در فصل سوم، معماری سامانه، ساختار بسته شواهد، سازوکار پاک سازی کد، سیاست تطبیق یافته ها، معیارهای ارزیابی، زیرساخت تکرارپذیری و در بخش پایانی، هویت مدل زبانی، ارائه دهنده و پیکربندی کامل اجرا تشریح می شود. در فصل چهارم، طراحی آزمایش ها و چهار پیکربندی مقایسه ای، نتایج به دست آمده، تحلیل قیف تصمیم عامل، تحلیل به ازای نوع آسیب پذیری، بررسی در مقیاس کامل، ارزیابی موتور شواهد قطعی روی محک پایتون و روی توابع دیویگن، ساختار هشدارها و تحلیل خطاها، بازه های اطمینان و حساسیت به اندازه نمونه، تحلیل هزینه و زمان، تفکیک زبانی و تحلیل کیفی یک تصمیم ارائه می شود. در فصل پنجم، نتیجه گیری، پاسخ پنج پرسش پژوهش، محدودیت ها و کارهای آینده بیان می شود.

فصل دوم

مبانی نظری و کارهای پیشین

۲.۱ تحلیل ایستا و مفهوم هشدار نادرست

تحلیل ایستا مجموعه‌ای از روش‌های خودکار است که هدف آن، یافتن خطاها و نقض‌های قرارداد در برنامه بدون اجرای آن است. این روش‌ها بر پایه ساخت یک نمایش میانی از برنامه، مانند گراف کنترل جریان یا درخت نحو انتزاعی، و سپس اعمال قواعد تحلیل روی آن نمایش استوارند. دو خانواده اصلی از تحلیل ایستا عبارت‌اند از: تحلیل مبتنی بر قواعد^۱ که با الگوهای از پیش تعریف‌شده الگوهای بد، مانند استفاده از فراخوانی‌های خطرناک، کار می‌کند؛ و تحلیل مبتنی بر تفسیر انتزاعی^۲ که با تقریب زدن مجموعه مقادیر ممکن یک متغیر در هر نقطه برنامه، به صورت ایستا به رفتار برنامه نزدیک می‌شود. این دو خانواده، مبنای تقریباً همه ابزارهای تجاری و متن‌باز امروزی هستند.

هر ابزار تحلیل ایستا زمانی ارزشمند است که بتواند میان سه دسته هشدار تمایز بگذارد: هشدار درست، هشدار نادرست و از دست رفتن آسیب‌پذیری واقعی. هشدار درست یعنی محل گزارش‌شده واقعاً یک نقص امنیتی دارد. هشدار نادرست یعنی محل گزارش‌شده ایرادی ندارد؛ برای نمونه مسیر داده هشدار داده‌شده هرگز اجرا نمی‌شود، یا اندازه بافر پیش از استفاده کنترل شده است. از دست رفتن آسیب‌پذیری واقعی یعنی ابزار ساکت مانده است، در حالی که کد آسیب‌پذیر است. اندازه‌گیری صرفاً دقت و یادآوری، بدون توجه به این‌که این معیارها با چه دانه‌بندی و با چه سیاستی محاسبه شده‌اند، مقایسه ابزارها را گمراه‌کننده می‌کند؛ از این رو در فصل چهارم دانه‌بندی معیارها به صراحت تعریف و در همه آزمایش‌ها به طور یکسان اعمال می‌شود.

بررسی‌های پیشین نشان می‌دهد که هشدار نادرست و شیوه ارائه هشدارها از موانع اصلی به کارگیری ابزارهای تحلیل ایستا هستند [۶]. همچنین تجربه صنعتی نشان داده است که بررسی دستی هزاران هشدار، هزینه قابل توجهی دارد و اعتماد توسعه‌دهندگان به ابزار را کاهش می‌دهد [۷].

۲.۲ ابزارهای تحلیل ایستای به کاررفته در این پژوهش

در فاز یک سامانه پیشنهادی، برای هر زبان، ابزارهای متن‌باز مناسب به صورت موازی روی هر فایل اجرا می‌شوند. برای پایتون از بندیت و برای زبان‌های C/C++ از دو ابزار مجزا استفاده می‌شود. انتخاب این ابزارها بر پایه تنوع رویکرد و پوشش زبان‌های دامنه پژوهش انجام شده است.

هر یک از این ابزارها نقاط قوت و ضعف مختص به خود را دارند. به عنوان نمونه، ابزار `cppcheck` با رویکردی قاعده‌محور، تحلیلی سریع و کم‌هزینه ارائه می‌دهد، اما محدودیت اصلی آن این است که عمق تحلیل نسبتاً کمی دارد. از سوی دیگر، `flawfinder` با تمرکز بر پوشش سبک‌های خطرناک برنامه‌نویسی، رویکرد قاعده‌محور دیگری را اتخاذ کرده است، اما این گستردگی باعث می‌شود به شدت پرهشدار باشد و سیگنال‌های کاذب زیادی تولید کند. ابزار `clang`

^۱ Rule-based

^۲ Abstract Interpretation

[۳] با تکیه بر رویکرد تحلیل مسیر، امکان مسیریمایی واقعی را فراهم می‌کند، اما نقطه‌ضعف آن وابستگی شدید به فرآیند کامپایل است. در نهایت برای ارزیابی کدهای پایتون، ابزار bandit با استفاده از الگوهای امنیتی، دامنه زبان پایتون را به‌خوبی پوشش می‌دهد، اما هشدارهای آن عمدتاً از نوع الگویی و استاتیک است. خلاصه‌ای از این ویژگی‌ها در جدول ۲.۱ دسته‌بندی شده است.

از میان این چهار ابزار، سه ابزار bandit، cppcheck و flawfinder در پیاده‌سازی نهایی یکپارچه شده‌اند. تحلیل‌گر ایستای clang نیز بررسی شد، اما چون در حالت تک‌فایلی به فرآیند کامپایل و پیکربندی پروژه وابسته است و نمی‌توان آن را به‌صورت قابل‌اتکا روی پرونده‌های منفرد اجرا کرد، در فاز یک به کار گرفته نشد. این محدودیت در فصل پنجم به‌عنوان یکی از کارهای آینده ذکر شده است.

ابزار	رویکرد	نقطه قوت	نقطه ضعف
cppcheck	قاعده‌محور	سریع و کم‌هزینه	عمق تحلیل کم
flawfinder	قاعده‌محور	پوشش سبک‌های خطرناک	بسیار پرهشدار
clang	تحلیل مسیر	مسیریمایی واقعی	وابسته به کامپایل؛ یکپارچه نشد
bandit	الگوی امنیتی پایتون	پوشش دامنه پایتون	هشدار الگویی

جدول ۲.۱: ابزارهای تحلیل ایستای ارزیابی‌شده برای فاز یک

مزیت اجرای موازی ابزارهای هر زبان، پوشش بیشتر و کاهش وابستگی به نقاط کور یک ابزار واحد است. عیب آن، تولید هشدارهای همپوشان است؛ به‌عنوان مثال، ممکن است چند ابزار یک سرریز بافر واحد را با چند شناسه متفاوت گزارش کنند. این هشدارهای تکراری، اگر تفکیک نشوند، هم آمار هشدار نادرست را بدتر نشان می‌دهند و هم بار کاری عامل جستجوگر را چند برابر می‌کنند. به همین دلیل، در فاز یک، یافته‌های همپوشان با استفاده از اثر انگشت و تطبیق خط در یک گروه ادغام می‌شوند.

۲.۳ تحلیل ایستا-پویا و رویکردهای ترکیبی

تحلیل پویا، برخلاف تحلیل ایستا، برنامه را اجرا می‌کند و رفتار واقعی آن را مشاهده می‌نماید. این رویکرد دقت بالایی برای خطاهایی دارد که در مسیر اجرایی رخ می‌دهند، اما تنها بخش کوچکی از کد را پوشش می‌دهد، زیرا اجرای همه حالت‌های ممکن یک برنامه، به‌طور کلی ناممکن است. رویکردهای ترکیبی^۲ تلاش می‌کنند این دو مزیت را هم‌زمان به دست آورند؛ برای نمونه، با استفاده از تحلیل ایستا برای یافتن محل‌های مشکوک و سپس اجرای هدفمند برنامه در همان محل‌ها. سامانه codey-security در این پژوهش نیز در همین خانواده مفهومی قرار می‌گیرد، با این تفاوت که به‌جای تحلیل پویا، از دو عامل زبانی مستقل به‌عنوان پیشنهاددهنده و داور مبتنی بر شواهد استفاده می‌کند.

^۲ Hybrid Analysis

۲.۴ مدل‌های زبانی بزرگ و عامل‌های نرم‌افزاری

مدل‌های زبانی بزرگ، مدل‌های آماری زبانی با توانایی تولید متن بر اساس پیش‌بینی توکن بعدی هستند که با مقیاس‌دهی حجم داده و پارامتر آموزش داده می‌شوند. این مدل‌ها در چند سال اخیر توانسته‌اند در وظایفی مانند تولید کد، خلاصه‌سازی و تحلیل ایستا مورد استفاده قرار گیرند [۱۱].

عامل نرم‌افزاری^۴ به برنامه‌ای اطلاق می‌شود که با دریافت یک هدف، به‌طور خودکار برنامه‌ریزی می‌کند، ابزار فراخوانی می‌کند و تا رسیدن به پاسخ، چرخه مشاهده-تصمیم-اقدام را تکرار می‌کند. در این پژوهش، عامل با یک نقش محدود تعریف شده است: پیشنهاد یک یافته مشخص یا داوری درباره همان یافته بر پایه شواهد ارائه‌شده. این محدودیت عامدانه است، زیرا جستجوگر باید در برابر وسوسه اثبات یافته مقاوم باشد و ارزیاب باید در برابر وسوسه تولید یافته جدید مقاوم باشد.

۲.۵ استدلال مبتنی بر شواهد

یکی از محدودیت‌های شناخته‌شده مدل‌های زبانی این است که می‌توانند پاسخی روان، مرتبط و در ظاهر مستند تولید کنند، در حالی که هیچ‌یک از اجزای آن پاسخ در متن ورودی موجود نبوده است. این پدیده که ریشه در نبود پیوند میان توکن‌های تولیدشده و جهان واقعی دارد، در ادبیات هوش مصنوعی با مسئله زمینه‌مندی^۵ توصیف شده است [۱۴].

راهکاری که در این پژوهش به کار گرفته شده، محدودکردن عامل به داوری بر پایه شواهد است. عامل موظف است تنها از محتوای بسته شواهد استفاده کند، در صورت کمبود شاهد، عدم قطعیت را اعلام نماید و به جای استنتاج آزاد، به قطعه کد و ساختار ارائه‌شده استناد کند. این رویکرد با خانواده روش‌های بازیابی‌شده^۶ هم‌خانواده است، با این تفاوت که هدف، افزایش دانش مدل نیست، بلکه مهار آن و کاهش پاسخ‌های بی‌پایه است [۱۳].

۲.۶ مجموعه داده جولیت

مجموعه آزمون جولیت^۷ یک مجموعه داده مرجع برای ارزیابی ابزارهای تحلیل ایستا است که توسط مؤسسه ملی استاندارد و فناوری آمریکا به‌عنوان بخشی از پایگاه آزمون ارزیابی امنیت نرم‌افزار منتشر شده است [۴]. این مجموعه برای هر نوع آسیب‌پذیری، جفت‌هایی از نسخه آسیب‌پذیر و نسخه سالم از یک سناریوی واحد تولید می‌کند. تنها تفاوت میان نسخه آسیب‌پذیر و سالم، در نقطه‌ای از جریان داده است؛ برای مثال، در یک سناریوی سرریز عدد صحیح، نسخه سالم همان

^۴ Software Agent

^۵ Grounding

^۶ Retrieval-Augmented Generation

^۷ Juliet Test Suite

محاسبه را با یک شرط کنترلی همراه می‌کند (نمونه‌ای از نسخه آسیب‌پذیر و نسخه سالم متناظر آن در یک سناریوی جولیت، به‌ترتیب در شکل‌های ۲.۱ و ۲.۲ نشان داده شده است).

CWE190_Integer_Overflow__int_rand_add_73b_bad.cpp

```
void badSink(list<int> dataList)
{
    int data = dataList.back();
    {
        /* POTENTIAL FLAW: Adding 1 to data could
           cause an overflow */
        int result = data + 1;
        printIntLine(result);
    }
}
```

شکل ۲.۱: نسخه آسیب‌پذیر یک سناریوی جولیت

CWE190_Integer_Overflow__int_rand_add_73b_good.cpp

```
void goodB2GSink(list<int> dataList)
{
    int data = dataList.back();
    /* FIX: Add a check to prevent an overflow */
    if (data < INT_MAX) {
        int result = data + 1;
        printIntLine(result);
    }
}
```

شکل ۲.۲: نسخه سالم همان سناریو؛ تنها تفاوت، یک شرط کنترلی است

ویژگی جفت‌بودن نسخه‌ها، هم فرصت و هم تهدید است. فرصت از آن رو که هر دو نسخه زمینه تقریباً یکسانی دارند، بنابراین یک هشدار نادرست را می‌توان به‌سادگی به توضیحی بیرونی نسبت داد. تهدید از آن رو که اگر مرزبندی داده‌ها درست انجام نشود، نسخه‌های مرتبط یک سناریو می‌توانند در دو بخش آموزش و آزمون قرار گیرند و نتیجه ارزیابی به‌شدت خوش‌بینانه شود. به همین دلیل، در این پژوهش برای هر نمونه یک شناسه گروه تعریف شده است تا نمونه‌های هم‌خانواده در یک بخش قرار گیرند.

علاوه بر جولیت، دو مجموعه دیگر برای پوشش زبان پایتون و کد واقعی به کار رفت. نخست، یک محک مصنوعی جفتی شامل ۴۸ فایل پایتون است: ۲۴ سناریوی آسیب‌پذیر و نسخه اصلاح‌شده همان سناریو. این محک برای سنجش مهار هشدار نادرست ساخته شده و جایگزین مجموعه واقعی نیست. دوم، مجموعه دیویگن است که در نسخه به‌کاررفته شامل ۲۷۰۲۵۸ تابع واقعی و برچسب‌دار C از پروژه‌های متن‌باز است؛ برخلاف توصیف اولیه در پروپوزال، این نسخه صرفاً توابع کرنل لینوکس نیست. خوانش برچسب‌های سارد و بیگ‌وال در پیاده‌سازی پشتیبانی می‌شود، اما چون نسخه معتبر و قابل بازیابی آن‌ها در این پژوهش در دسترس نبود، عددی از آن‌ها گزارش نمی‌شود.

۲.۷ کارهای پیشین

کارهای پیشین را می‌توان در سه دسته کلی تحلیل کرد. دسته نخست، ارزیابی‌های انتقادی ابزارهای تحلیل ایستا هستند. این پژوهش‌ها نشان می‌دهند موانعی مانند حجم بالای هشدارهای نادرست و شیوه نامناسب ارائه این هشدارها، پذیرش و استفاده عملی از این ابزارها را در صنعت محدود کرده است. در این دسته، سنجش واقع‌بینانه‌ای از ابزارها ارائه شده که نشان می‌دهد عملکرد آن‌ها به‌شدت به مجموعه داده وابسته است، و همین امر نقش مهمی در ایجاد مبانی تعریف معیارها ایفا کرده است.

دسته دوم، به کارگیری روش‌های یادگیری ماشین برای تشخیص آسیب‌پذیری اختصاص دارد. رویکردی مانند مدل دیویگن توانسته است با یادگیری معنای برنامه روی تابع‌های واقعی زبان C، بهبود چشمگیری در افزایش دقت نسبت به روش‌های سنتی گزارش کند. با این حال، محدودیت اساسی این روش‌ها این است که معمولاً مبانی نظری روشی برای توضیح دلیل هر هشدار ارائه نمی‌کنند و در برابر کد ناآشنا توانایی تعمیم‌پذیری پایینی دارند.

دسته سوم، بر کاربرد مدل‌های زبانی بزرگ در تحلیل کد متمرکز است. در این دسته، مدل‌های زبانی با استفاده از قابلیت فهم عمیق زبان طبیعی، به کمک ابزارهای ایستا آمده‌اند و نشان داده‌اند که توانایی تولید کد، به‌تنهایی تضمینی برای ایمنی کد نیست. با این وجود، چالش اساسی در تلفیق این مدل‌ها با تحلیل ایستا، بروز خطای توهم (Hallucination) و عدم قطعیت در نتایج خروجی مدل است. خلاصه‌ای از این مقایسه‌ها در جدول ۲.۲ ارائه شده است.

دسته	نمونه	نقش	محدودیت	جایگاه این کار
ارزیابی انتقادی	[۶] [۸]	سنجش واقع‌بینانه	وابسته به مجموعه داده	مبانی تعریف معیار
یادگیری ماشین	[۹]	دقت بالاتر	بدون توضیح‌پذیری	مقایسه عددی
مدل زبانی	[۱۰] [۱۱]	فهم زبان طبیعی	توهم و عدم قطعیت	تلفیق با ایستا

جدول ۲.۲: جایگاه کار پیشنهادی در میان کارهای پیشین

۲.۸ جایگاه کار پیشنهادی

سامانه codey-security در این پژوهش از دسته سوم متأثر است، اما بر یک نکته روش‌شناختی تأکید دارد که در بسیاری از کارهای موجود رعایت نمی‌شود: ارزیابی کمی باید با تعریف صریح معیارها و کنترل نشت برچسب همراه باشد. همچنین، به جای جایگزینی کامل ابزارهای ایستا، این پژوهش آن‌ها را صرفاً به عنوان تأمین‌کننده نامزد به کار می‌گیرد و از عامل‌های زبانی تنها برای پیشنهاد و پالایش شواهد بهره می‌گیرد. این انتخاب هوشمندانه، امکان گزارش یک نقطه کاری قابل تنظیم را فراهم می‌کند: سامانه می‌تواند با تغییر آستانه تأیید، روی نقطه‌ای از منحنی مبادله دقت و یادآوری بنشیند که متناسب با نیازها و هزینه‌های هر سازمان باشد.

۲.۹ چالش‌های ارزیابی سامانه‌های مبتنی بر مدل زبانی

بخش‌های پیشین نشان داد که مدل‌های زبانی می‌توانند در کنار تحلیل ایستا مفید باشند، اما درباره اینکه یک چنین تلفیقی واقعاً چقدر خوب است، پاسخ روشنی نمی‌دهند. دلیل این امر، فراتر از کیفیت خود مدل‌ها، در دشواری خود ارزیابی است. تجربه این پژوهش نشان می‌دهد که این دشواری‌ها را می‌توان در پنج محور دسته‌بندی کرد؛ محورهایی که بسیاری از پژوهش‌های موجود یا به آن‌ها نمی‌پردازند یا تنها یکی دو مورد از آن‌ها را گزارش می‌کنند.

نخست، پایداری پاسخ است. یک سامانه داوری که خروجی آن در دو اجرای پیاپی متفاوت باشد، هیچ معیار قابل اتکایی تولید نمی‌کند؛ چرا که تفاوت ثبت‌شده دیگر قابل نسبت دادن به هیچ عامل مستقلی نیست. راهکار شناخته‌شده، پایین آوردن دما و ثابت نگه داشتن پیکربندی است، اما این راهکار هزینه خود را دارد: دمای پایین‌تر، تنوع پاسخ را کم می‌کند و ممکن است دامنه چیزهایی را که عامل می‌بیند نیز محدود کند. در این پژوهش، دمای 0.2 انتخاب شد پس از آنکه دمای $1/0$ یک بسته واحد را در دو اجرا نخست تأیید و سپس رد کرد.

دوم، هزینه محاسباتی و زمانی است که مستقیماً بر اندازه نمونه اثر می‌گذارد. در یک مدل استدلالی، بخش بزرگی از خروجی صرف استدلال پنهان می‌شود و این استدلال، پیش از دیده شدن هر پاسخی هزینه می‌تراشد. نتیجه عملی آن است که پژوهشگر ناچار است میان عمق هر بررسی و شمار نمونه‌ها مصالحه کند؛ و چون عدم قطعیت آماری معیارها تابع شمار نمونه است، هزینه مدل در نهایت به بازه اطمینان یافته‌ها سرایت می‌کند. پژوهشی که هزینه خود را گزارش نکند، در واقع خواننده را از داوری درباره کافی بودن اندازه نمونه باز می‌دارد.

سوم، نشت برچسب در خود مجموعه داده‌های مرجع است. بسیاری از این مجموعه‌ها، برچسب را در نام فایل، در حاشیه‌نویسی کد یا در ساختار خود سناریو حمل می‌کنند. اگر این متن عیناً در اختیار مدل قرار گیرد، اندازه‌گیری دیگر توانایی تحلیل کد را نمی‌سجد، بلکه توانایی خواندن برچسب را می‌سجد؛ و بدتر آنکه نتیجه، بسیار بهتر از واقعیت به نظر می‌رسد و هیچ خطایی در آن آشکار نمی‌شود. کنترل این نشت، پیش‌شرط هر ارزیابی معنادار است، نه یک بهینه‌سازی.

چهارم، شفافیت پیکربندی است. گزارش تنها نام یک مدل، بدون نشانی سرویس، بدون دما، بدون سقف استدلال، بدون بودجه توکن و بدون سیاست بازتلاش، نتیجه‌ای تولید می‌کند که توسط پژوهشگر دیگر قابل بازتولید نیست. این کمبود، مقایسه میان پژوهش‌ها را عملاً ناممکن می‌کند: دو عدد ظاهراً همشکل، ممکن است حاصل دو پیکربندی کاملاً متفاوت باشند. ثبت کامل پیکربندی در فراداده هر نتیجه، راه برون‌رفت از این وضعیت است و در فصل سوم به تفصیل آمده است.

پنجم، پایداری سرویس‌دهنده است. قطعی شبکه، زمان متوقف و پاسخ ناقص در یک ارزیابی زبانی، برخلاف یک خطای محاسباتی محلی، خودش را به شکل «آسیب‌پذیری پیدا نشد» نشان می‌دهد؛ یعنی همان پیامدی را دارد که یک ابزار ضعیف دارد. اگر نمونه‌های خطادار شمرده نشوند یا گزارش نشوند، یادآوری گزارش شده به جای آنکه ویژگی روش باشد، ویژگی سرویس آن لحظه می‌شود. به همین دلیل، تفکیک صریح نمونه‌های خطادار از نمونه‌های سالم بدون هشدار، در این پژوهش به عنوان یک ضرورت روش‌شناختی در نظر گرفته شده است.

این پنج محور در شکل ۲.۳ خلاصه شده است. نکته ساختاری آن‌ها آن است که هیچ‌یک به خود مدل مربوط نیست؛ همه به شیوه ارزیابی مدل مربوط‌اند. بنابراین، یک پژوهش که تنها دقت و یادآوری گزارش کند، در واقع از زیر بار دست‌کم پنج پرسش روش‌شناختی گذشته است.

چالش ۱	چالش ۲	چالش ۳	چالش ۴	چالش ۵
پایداری پاسخ	هزینه محاسباتی	نشت برچسب	شفافیت پیکربندی	پایداری سرویس
خروجی ثابت در دو اجرا	نمونه، بازه اطمینان	خواندن پاسخ، نه کد	بازتولید ناممکن نتیجه	خطای شبکه، یادآوری کاذب

شکل ۲.۳: پنج محور چالش‌برانگیز ارزیابی سامانه‌های مبتنی بر مدل زبانی

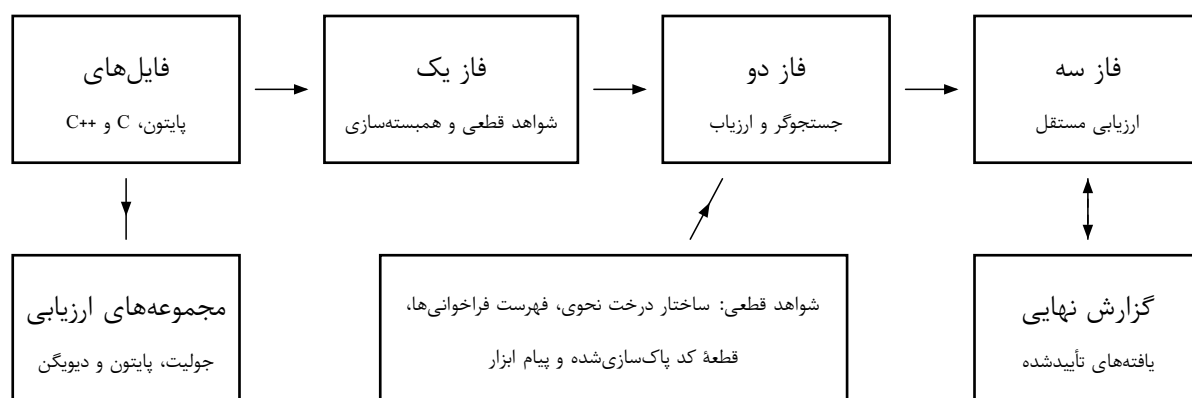
جمع‌بندی این پنج محور آن است که یک ارزیابی قابل اتکا از تلفیق مدل زبانی با تحلیل ایستا، تنها زمانی شکل می‌گیرد که ارزیابی خود ابزار ارزیابی نیز انجام شده باشد: پایداری پاسخ سنجیده شده باشد، هزینه گزارش شده باشد، نشت برچسب کنترل شده باشد، پیکربندی کامل ثبت شده باشد و خطاهای سرویس از خطاهای روش تفکیک گردیده باشد. سه بخش نخست فصل چهارم، یعنی ساختار خطاها، بازه‌های اطمینان و تحلیل هزینه، دقیقاً پاسخ همین پنج محورند و فصل پنجم نیز پیامدهای باقی‌مانده را باز می‌گوید.

فصل سوم

طراحی و پیاده‌سازی سامانه

۳.۱ معماری کلی

سامانه پیشنهادی از سه فاز متوالی تشکیل شده است. فاز یک، شواهد قطعی است و مسئولیت آن تولید فهرستی از یافته‌های خام ابزارها، استخراج ساختار نحوی، بازیابی مسیر منبع تا نقطه مصرف و همبسته‌سازی یافته‌های تکراری است. فاز دو، داوری چندعامله است: عامل جستجوگر فرضیه‌های آسیب‌پذیری را پیشنهاد می‌کند و عامل ارزیاب هر فرضیه را جداگانه بررسی و تأیید یا رد می‌کند. فاز سه، ارزیابی مستقل است و پیش‌بینی‌ها را با همان سیاست تطبیق و معیارها در برابر حقیقت زمین می‌سجد. این تفکیک فازها عمدی است: هر فاز یک پرسش مشخص را پاسخ می‌دهد و خروجی آن قابل ارزیابی جداگانه است، بنابراین می‌توان اثر هر فاز را مستقل از دو فاز دیگر سنجید.



شکل ۳.۱: معماری سه‌فاز سامانه و جریان داده میان فازها

جداسازی فازها یک مزیت عملی نیز دارد. اجرای ابزارهای ایستگاه هزینه پردازشی محلی دارد، در حالی که فراخوانی مدل زبانی هزینه زمانی و شبکه‌ای دارد. جدا بودن این دو، امکان اجرای موازی آن‌ها را روی مجموعه‌ای از نخ‌ها و یک زمان‌بند پرتاب^۱ می‌دهد، بدون آنکه فراخوانی‌های زبانی در انتظار، اجرای ابزارهای ایستگاه را متوقف کنند.

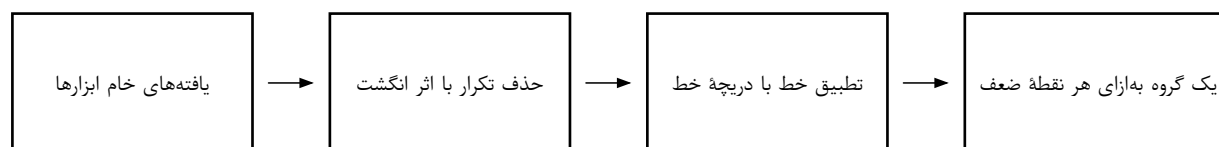
^۱ Scheduler

۳.۲ فاز یک: تحلیل ایستا

در فاز یک، برای هر فایل، ابزارهای متن‌باز متناسب با زبان آن اجرا می‌شوند. بندیت برای پایتون و سی‌پی‌پی‌چک و فلاپایندر برای C و C++ به کار گرفته می‌شوند. خروجی خام هر ابزار به فهرستی ساخت‌یافته از یافته‌ها تبدیل می‌شود. برای هر یافته، شناسهٔ ابزار، شمارهٔ خط، شدت، پیام و در صورت وجود، شناسهٔ CWE ذخیره می‌شود. انتخاب چند ابزار به جای یک ابزار، به دلیل پوشش مکمل آن‌هاست: هیچ‌یک به تنهایی همهٔ سناریوهای مجموعهٔ ارزیابی را پوشش نمی‌دهد. هم‌زمان، تجزیه‌گر درخت نحوی [۱۲] توابع، فراخوانی‌ها، انتساب‌ها، ارجاع‌ها و واردسازی‌ها را استخراج می‌کند و موتور بازیابی مسیر، مسیرهای نامزد منبع تا نقطهٔ مصرف را در محدودهٔ همان تابع می‌سازد.

۳.۳ همبسته‌سازی یافته‌ها

خروجی خام ابزارها سه مشکل دارد که پیش از بازیابی باید برطرف شود. نخست، یافته‌های تکراری است؛ چند ابزار ممکن است یک نقص واحد را گزارش کنند. دوم، گزارش‌های ناهمگون است؛ پیام‌ها قالب یکسانی ندارند و برخی تنها یک قاعدهٔ سبک را گزارش می‌کنند. سوم، یافته‌های پراکنده‌اند؛ یک نقطهٔ ضعف واحد ممکن است در چند خط و با چند شناسهٔ متفاوت ظاهر شود.



شکل ۳.۴: زنجیرهٔ ادغام یافته‌های تکراری و تشکیل گروه‌های بازیابی

برای رفع این سه مشکل، سه گام متوالی تعریف شده است. گام نخست، حذف تکرار با استفاده از یک اثر انگشت شامل نام فایل، نوع آسیب‌پذیری و خط است. گام دوم، تطبیق خط با یک دریچهٔ خط است، به این معنا که دو یافته در فاصلهٔ کمتر از پنج خط از یکدیگر، مربوط به یک نقطهٔ ضعف واحد تلقی می‌شوند. گام سوم، ساخت گروه بازیابی است؛ هر گروه، مجموعهٔ یافته‌های ادغام‌شده به همراه فهرست ابزارهای گزارش‌دهندهٔ آن است. هدف از این ادغام، کاهش بار کاری عامل جستجوگر و جلوگیری از چند بار پرسش دربارهٔ یک نقطهٔ ضعف واحد است.

۳.۴ فاز دو: دآوری چندعامله

در فاز دو، عامل جستجوگر ابتدا همه فایل را می‌بیند و فهرستی از فرضیه‌های آسیب‌پذیری، شامل شناسه مشکوک CWE، خط مورد بازبینی و توصیف یک جمله‌ای منبع و نقطه مصرف، تولید می‌کند. سپس عامل ارزیاب هر فرضیه را جداگانه بررسی می‌کند و باید یکی از سه تصمیم جدول ۳.۱ را برگرداند. دو نکته در این طراحی قابل تأمل است. نخست، وجود گزینه نامطمئن، عامل را از دوگزینه‌ای بودن رها می‌کند و اجازه می‌دهد در نبود شاهد کافی، صریحاً عدم قطعیت خود را اعلام نماید؛ خروجی نامطمئن در گزارش نهایی به‌عنوان نیازمند بررسی انسانی حفظ می‌شود. دوم، ارزیاب استدلال جستجوگر را نمی‌بیند و تنها فرضیه را دریافت می‌کند؛ این تفکیک، اثر لنگرانداختن استدلال اولیه بر دآوری نهایی را کاهش می‌دهد. تنها خروجی ارزیاب می‌تواند به یافته نهایی تبدیل شود و همه تأییدها، رد‌ها و موارد نامطمئن در گزارش تصمیم ثبت می‌شوند.

خروجی	معنا	اثر بر گزارش
تأییدشده	CONFIRMED	شاهد کافی برای پذیرش یافته
ردشده	REJECTED	شاهد ناسازگار با یافته
نامطمئن	UNCERTAIN	شاهد ناکافی
		نیازمند بررسی انسانی

جدول ۳.۱: سه خروجی ممکن عامل ارزیاب

۳.۵ بسته شواهد

کیفیت تصمیم عامل، مستقیماً تابع کیفیت شواهدی است که در اختیارش قرار می‌گیرد. بسته شواهد جستجوگر شامل اطلاعات پایه از جمله نقش، زبان، فایل، زمینه کد، یافته‌های ابزارهای ایستا، خلاصه ساختاری و در نهایت مسیرهای نامزد منبع تا نقطه مصرف است. در مقابل، بسته‌ای که برای ارزیاب ارسال می‌شود تنها بر یک فرضیه متمرکز است و شامل همان فرضیه، زمینه کد پیرامون خط مشکوک، مسیرهای نزدیک به آن فرضیه و یافته‌های نزدیک ابزارهاست؛ به‌طوری‌که استدلال و تحلیل جستجوگر به‌هیچ‌وجه در آن گنجانده نمی‌شود تا استقلال ارزیاب حفظ گردد. موتور قطعی سیستم نیز موظف است مسیر منبع تا نقطه مصرف را دقیقاً در محدوده همان تابع بازبایی کند. برای تک‌تک این مسیرها، اطلاعات جامعی نظیر عبارت منبع، عبارت نقطه مصرف، شماره خط‌ها، گام‌های میانی، منشأ آلودگی و همچنین (در صورت وجود) قانون کاهش خطر در بسته ثبت و بایگانی می‌شود.

evidence-packet.json

```
{
  "role": "scanner",
  "language": "python",
  "file": "example.py",
  "source_context": {
    "start_line": 4,
    "end_line": 12,
    "snippet": "    6: name = sys.argv[1]\n ..."
  },
  "static_tool_findings": [
    { "tool": "bandit", "message": "possible shell injection" }
  ],
  "structural": {
    "functions": [ { "name": "run_user_command", "line": 4 } ],
    "calls": [ { "callee": "subprocess.call", "line": 6 } ]
  },
  "dataflow_chains": [
    {
      "source": "sys.argv[1]",
      "sink": "subprocess.call(command, shell=True)",
      "mitigated": false
    }
  ]
}
```

شکل ۳.۲: نمونه ساده‌شده بسته شواهد جستجوگر

قطعه کد ارسالی با شعاع هشت خط پیش و پس از خط مورد نظر ساخته می‌شود. این مقدار، مصالحه‌ای دقیق میان مدیریت هزینه توکن و کفایت زمینه است؛ به عبارتی شعاع کمتر باعث می‌شود تعریف متغیر یا نقطه کنترلی مرتبط از قطعه بیرون بماند، در حالی که شعاع بیشتر تنها موجب پراکندگی توجه مدل می‌گردد. شواهد ساختاری نیز با استفاده از تجزیه‌گر درخت نحوی تولید می‌شود و به عامل این امکان را می‌دهد که به جای تکیه بر حدس و گمان، مستقیماً فهرست فراخوانی‌های همان تابع را رویت کند. در فصل چهارم به‌وضوح نشان داده می‌شود که افزودن این بخش ساختاری، تعداد تأییدهای عامل را به‌شکل معناداری افزایش می‌دهد. اعتماد ارزیاب نیز در این فرآیند به‌صورت کاملاً قطعی مهار می‌شود: هرگونه تأیید کم‌اعتماد بلافاصله تنزل می‌یابد، ادعای تأیید زنجیره باید الزاماً با رأی ارزیاب سازگار باشد، و نهایتاً برای دسته‌هایی که موتور، مدل‌سازی می‌کند، نبود مسیر واقعی در فایل به رد قطعی یافته می‌انجامد.

۳.۶ پاک‌سازی کد و جلوگیری از نشت برچسب

مجموعه داده جولیت برچسب واقعی هر فایل را درون خود متن فایل قرار می‌دهد. برای نمونه، در فایل‌های آسیب‌پذیر، حاشیه‌نویسی‌هایی با عبارت‌هایی نظیر نشانگر وجود نقص و شماره CWE وجود دارد و نام فایل و نام اشیا نیز شامل واژه بد یا خوب هستند. اگر این متن عیناً به مدل داده شود، مدل می‌تواند بدون هیچ تحلیلی پاسخ درست را از روی نام فایل حدس بزند و نتیجه آزمایش، اندازه‌گیری توانایی تحلیل نخواهد بود، بلکه صرفاً اندازه‌گیری توانایی خواندن برچسب است.

برای جلوگیری از این نشت اطلاعاتی خطرناک، سه اقدام متوالی در ماژول پاک‌سازی سیستم انجام می‌شود. نخست، محتوای حاشیه‌نویسی‌ها در جای خود به‌طور کامل با فاصله جایگزین می‌شود، به‌گونه‌ای که تعداد خط‌ها و شماره خط گزارش‌شده توسط ابزارها کوچک‌ترین تغییری نکند. دوم، شناسه‌های افشاگر (مانند نام‌هایی که شامل واژه بد یا خوب هستند) طی یک فرآیند ایزوله، به نام‌هایی پایدار و کاملاً بی‌معنا نگاشت می‌شوند؛ به این ترتیب که یک نام چه در قطعه کد، چه در شواهد ساختاری و چه در پیام ابزار، همواره به یک شکل ثابت و خنثی ظاهر شود. سوم، عملیات پاک‌سازی روی تمام فیلدهای متنی بسته شواهد اعمال می‌شود و صرفاً به قطعه کد محدود نیست، زیرا پیام ابزارها و نام فایل نیز به‌سادگی می‌توانند برچسب پنهان را افشا کنند.

CWE122_Heap_Based_Buffer_Overflow__c_CWE805_struct_memcpy_82a_bad.cpp

```

24:
25: void sym_1902e3()
26: {
27:     twoIntsStruct * data;
28:     data = NULL;
29:
30:     data = (twoIntsStruct *)malloc(50*sizeof(twoIntsStruct));
31:     if (data == NULL) {exit(-1);}
32:     sym_9623a0* baseObject = new sym_5b7d8f;
33:     baseObject->action(data);
34:     delete baseObject;
35: }
```

شکل ۳.۳: قطعه کد پس از پاک‌سازی؛ حاشیه‌نویسی‌ها محو و شناسه‌های افشاگر بازنام‌گذاری شده‌اند

یک استثنا عمدی در این ماژول وجود دارد: ارجاع‌هایی که قالب و شکل CWE-122 دارند بازنویسی نمی‌شوند، زیرا سازوکار تطبیق یافته‌ها در مراحل بعدی مستقیماً به آن‌ها نیاز دارد. همچنین، برای اطمینان از پایداری رفتار سامانه، آزمون‌های خودکاری نوشته شده است که در صورت بازگشت نشت برچسب، آزمایش فوراً شکست می‌خورد؛ زیرا در

آن صورت همه آزمایش‌های مبتنی بر مدل زبانی به‌طور خاموش از سنجش تحلیل کد، به سنجش نشت اطلاعات تبدیل می‌شوند.

۳.۷ سیاست تطبیق یافته‌ها و معیارها

سنجش خروجی سامانه، نیازمند تصمیم قطعی درباره این مسئله است که یک هشدار چه زمانی دقیقاً با یک آسیب‌پذیری واقعی منطبق دانسته شود. در این سامانه، سه قانون اصلی به کار می‌رود که در جدول ۳.۲ خلاصه شده‌اند.

نخستین قانون، الزام هم‌بودن فایل است. این شرط تضمین می‌کند که تطبیق، منحصرأ در سطح پروژه و فایل‌های مرتبط واقعی انجام گرفته است و بدون آن، هیچ‌گونه تطبیقی توسط سامانه پذیرفته نمی‌شود. دومین قانون، استفاده از خانواده گسترده‌تر CWE به جای شناسه‌های نقطه‌ای است. این قانون از یک واقعیت عملی ناشی می‌شود: ابزارهای تحلیل ایستا اغلب شناسه والد آسیب‌پذیری را گزارش می‌کنند، در حالی که برچسب مجموعه‌داده، شناسه فرزند را نام می‌برد. برای نمونه، یک ابزار ممکن است سرریز بافر کلاسیک را گزارش کند، در حالی که نمونه در اصل متعلق به زیرشاخه سرریز بافر مبتنی بر حافظه چرخه‌ای است. بدون نگاشت خانواده و پذیرش هم‌پوشانی خانواده‌ها، این یافته‌های واقعی، هم‌زمان به‌عنوان هشدار نادرست و همچنین از دست رفتن آسیب‌پذیری شمرده می‌شوند که آمار ارزیابی را مخدوش می‌کند. سومین قانون، اعمال آستانه تحمل خط است که خطای گزارش ابزار را تا سقف فاصله کمتر از ۵ خط مجاز دانسته و آن را جبران می‌کند.

قانون تطبیق	شرط	دلیل
هم‌بودن فایل	نام فایل یکسان	تطبیق در سطح پروژه واقعی
خانواده CWE	هم‌پوشانی خانواده	ابزارها CWE والد گزارش می‌کنند
فاصله خط	کمتر از ۵ خط	خطای گزارش ابزار جبران می‌شود

جدول ۳.۲: سیاست تطبیق یافته‌ها با برچسب واقعی

چون یک هشدار ممکن است هم‌زمان با چند آسیب‌پذیری واجد شرایط هم‌پوشانی داشته باشد، جفت‌سازی نهایی به‌صورت حریصانه انجام می‌شود: همه نامزدها بر پایه امتیاز مرتب می‌شوند، هر نامزد تنها زمانی پذیرفته می‌شود که نه هشدار آن و نه آسیب‌پذیری متناظرش پیش‌تر مصرف شده باشد، و در امتیازهای برابر، ترتیب شاخص هشدار تعیین‌کننده است. از آنجا که هشدارها پیش از جفت‌سازی به‌ترتیب پایدار نام فایل، شماره خط و ابزار مرتب می‌شوند، این قاعده نتیجه قطعی و بازتولیدپذیر می‌دهد: بازمحاسبه معیارها روی همان گزارش‌های بایگانی‌شده، دقیقاً همان فهرست تطبیق‌ها را بازسازی می‌کند.

در مورد معیارها، یک تصمیم روش‌شناختی مهم گرفته شده است. معیارهای مثبت درست، مثبت نادرست و منفی نادرست همگی در دانه‌بندی «یافته» محاسبه می‌شوند، به این معنی که هر هشدار به‌صورت کاملاً جداگانه شمرده می‌شود. اما در مقابل، معیار منفی درست منحصرأ در دانه‌بندی «فایل» محاسبه می‌گردد؛ یعنی یک فایل سالم تنها زمانی به‌عنوان یک منفی درست تلقی می‌شود که مطلقاً هیچ هشدار در آن ثبت نشده باشد. دلیل این انتخاب استراتژیک آن است که هشدار نادرست، ماهیتاً پدیده‌ای در سطح فایل است: توسعه‌دهنده در عمل با فایلی روبه‌رو می‌شود که هشدار روی آن دریافت کرده است، نه صرفاً با یک خط منفرد و مجرد در خلأ. برای رفع هرگونه ابهام، نرخ هشدار روی فایل سالم نیز به‌طور جداگانه در نتایج گزارش می‌شود.

۳.۸ شمای گزارش و نمونه‌های واقعی

خروجی نهایی سامانه یک فایل گزارش ساخت‌یافته در قالب JSON است و هر یافته تأییدشده باید بدون استثنا همه فیلدهای مندرج در جدول ۳.۳ را در خود داشته باشد. مرجع مربوط به CVE کاملاً از یک فهرست دستی و کنترل‌شده پر می‌شود، نه از خروجی‌های متنی مدل؛ بنابراین امکان ندارد شناسه جعلی یا متوهمانه وارد گزارش نهایی سیستم شود. اگر برای یک دسته خاص، نمونه واقعی و منتخبی وجود نداشته باشد، گزارش تنها پیوند صفحه کلاس CWE را نمایش می‌دهد و به‌هیچ‌وجه جای خالی را با حدس زدن پر نمی‌کند.

جدول ۳.۳ الزامات پیشنهاد را در کنار محتوا و فیلدهای گزارش به دقت نگاشت کرده است؛ از جمله اختصاص فیلد cwe برای نوع آسیب‌پذیری، severity برای تحلیل شدت گزارش، و فیلدهای مسیر نظیر source و sink برای اثبات جریان شواهد.

فیلد گزارش	محتوا	الزام پیشنهاد
cwe	شناسه دسته ضعف	نوع آسیب‌پذیری
severity	شدت	تحلیل گزارش
file, line, function	فایل، خط و تابع	محل دقیق کد
explanation	توضیح فنی کوتاه	توضیح تحلیلی
source, sink, chain	عبارت‌ها، خط‌ها و مسیر بازیابی‌شده	شاهد جریان داده
chain_verified, taint_origin	تأیید زنجیره و منشأ آلودگی	شاهد جریان داده
related_cves, references	شناسه واقعی و پیوند مرجع	مرجع CVE
confidence, detected_by	اعتماد و نقش عامل‌ها	قابلیت حسابرسی

جدول ۳.۳: شمای گزارش نهایی سامانه

جدول ۳.۴ دامنه آسیب‌پذیری‌های پیشنهادی را با نمونه‌های واقعی استفاده‌شده در پیاده‌سازی سیستم مقایسه می‌کند. در فرآیند اجرا، مشخص شد که دو شناسه پیشنهادی در واقع نادرست بوده‌اند و در پیاده‌سازی نهایی با شناسه‌های دقیق‌تر جایگزین و اصلاح شدند. به عنوان مثال، به جای استفاده از CVE-2021-25239 که اساساً یک مسئله مرتبط با افشای اطلاعات در محصول ترندمیکرو است، نمونه بسیار مرتبط‌تر تزریق قالب جینجا یعنی CVE-2025-27516 به کار گرفته شد. همچنین از آنجا که CVE-2022-22817 به طور خاص به آسیب‌پذیری تزریق عبارت در کتابخانه پیلو اشاره دارد و ارتباطی به بازسازی ناامن در پای‌یامل ندارد، نمونه تأییدشده CVE-2017-18342 برای پوشش موضوع بازسازی ناامن به سیستم افزوده شد.

شناسه‌های نادرستی که در پیشنهاد ذکر شده بودند، در فهرست مرجع به صورت خارج از دامنه و با ذکر صریح دلیل عدم تطابق نگه داشته شده‌اند تا حذف آن‌ها از سیستم پنهان نمانده و شفافیت پژوهش حفظ شود. برای بررسی دقیق‌تر مراجع مربوط به هر شناسه نظیر CVE-2025-27516، CVE-2022-22817، CVE-2017-7529، CVE-2021-3156، CVE-2017-18342 و CVE-2021-25239 به ترتیب به مراجع [۱۶] تا [۲۱] مراجعه شود.

دسته	زبان	نمونه واقعی پیاده‌سازی	نسبت با پیشنهاد
CWE-122	C	CVE-2021-3156	همان نمونه پیشنهادی؛ مرجع ردهت CWE-122 و علت ریشه‌ای CWE-193 است
CWE-190	C	CVE-2017-7529	همان نمونه پیشنهادی
CWE-94	Python	CVE-2022-22817	همان شناسه پیشنهادی، ولی نمونه واقعی تزریق عبارت در پیلو است، نه بازسازی ناامن در پای‌یامل
CWE-1336	Python	CVE-2025-27516	جایگزین شناسه نادرست پیشنهادی برای تزریق قالب
CWE-502	Python	CVE-2017-18342	نمونه تأییدشده بازسازی ناامن در پای‌یامل؛ شناسه پیشنهادی متعلق به آن نیست
CWE-78	Python	بدون شناسه منتخب	تنها مرجع کلاس؛ همان دامنه پیشنهادی

جدول ۳.۴: دامنه پیشنهادی و نمونه‌های واقعی به کاررفته در گزارش

۳.۹ زیرساخت اجرا و تکرارپذیری

یکی از پیش‌نیازهای بنیادین برای اثبات هر ادعای کمی در محیط‌های دانشگاهی، تضمین تکرارپذیری آزمایش‌هاست. به همین دلیل، در پایان هر اجرای سامانه، در کنار نتایج خروجی، فراداده کاملی شامل اطلاعات مربوط به نسخه هر ابزار استفاده‌شده، نام دقیق و نسخه مدل زبانی، نشانی سرویس، شعاع تنظیم‌شده برای قطعه کد، میزان تحمل خط، سیاست‌های اعمال‌شده در تطبیق CWE و همچنین مدت زمان اجرای کامل فرآیند، همراه با نتایج ذخیره و بایگانی می‌شود. علاوه بر این، جریان اجرای بازبینی به‌گونه‌ای انعطاف‌پذیر پیاده‌سازی شده است که در صورت قطع شدن موقت پاسخ سرویس، در زمان اجرای دوباره سیستم، تنها مواردی که با شکست یا قطعی مواجه شده‌اند پردازش شوند. این ویژگی ساختاری، نه تنها خطاهای ناگزیر شبکه را از خطاهای روش‌شناختی سیستم به‌خوبی جدا می‌کند، بلکه هزینه گزاف ارزیابی‌های تکراری و سربار پردازش‌های بیهوده را نیز به‌طور چشمگیری کاهش می‌دهد.

جزئیات ثبت‌شده در این فراداده، دقیقاً همان چیزهایی است که بازتولید یک عدد را ممکن می‌کند: نسخه هر ابزار، یعنی Cppcheck 2.22.0 و flawfinder 2.0.20 و bandit 1.9.4، نسخه محیط اجرا، نشانی سرویس، نام مدل، هم‌زمانی، سقف گروه‌ها، خط تحمل و سیاست تطبیق. افزون بر این، گزارش‌های خام ابزارها در درون خود پرونده نتیجه نگه داشته می‌شوند؛ به همین دلیل محاسبه دوباره معیارها نه به فاز یک نیاز دارد و نه به فراخوانی دوباره مدل، بلکه تنها از روی همان گزارش‌های بایگانی‌شده بازسازی می‌شود و زمان و منبع این بازمحاسبه نیز در فراداده ثبت می‌گردد.

این ساختار یک مزیت روش‌شناختی دیگر نیز دارد: چون معیارها تابع خود گزارش‌ها هستند نه تابع اجرای سامانه، اصلاح یک قانون تطبیق معیوب می‌تواند بی‌آنکه حتی یک فراخوانی مدل تکرار شود، روی همه نتایج پیشین اعمال شود. در عمل همین مسیر به کار رفت: پس از اصلاح سیاست شمارش هشدارهای نادرست، همه معیارهای پیکربندی‌های A تا D تنها با بازمحاسبه از روی گزارش‌های بایگانی‌شده به‌روزرسانی شد؛ و پرونده ارزیابی ۶۰۰ نمونه‌ای نیز از روی همان گزارش‌ها و برچسب‌های جولیت بازسازی گردید، به‌گونه‌ای که پیش‌بینی‌های بایگانی‌شده با همان شمار نمونه‌ها، همان لایه‌بندی و همان تفکیک زبانی، دقیقاً همان معیارها را بازتولید می‌کنند. بازتولید پیش‌بینی‌ها از صفر نیز همین‌گونه ممکن است؛ هزینه مجدد تنها زمانی لازم می‌شود که خود پیش‌بینی‌ها از دست رفته باشند.

۳.۱۰ مدل زبانی، ارائه‌دهنده و پیکربندی اجرا

هر ادعای کمی درباره کیفیت بازیابی، در واقع ادعایی درباره یک مدل مشخص، با یک نشانی مشخص و با یک پیکربندی مشخص است؛ جابه‌جایی هر یک از این سه، عدد نهایی را تغییر می‌دهد. از این رو، گزارش نام مدل به‌تنهایی کافی نیست و باید پیکربندی کامل فراخوانی نیز در کنار نتایج آمده باشد. جدول ۳.۵ پیکربندی به‌کاررفته در همه آزمایش‌های فصل چهارم را می‌آورد.

پارامتر	مقدار	چرا این مقدار
model	mimo-v2.6-flash-free	شناسه مدل ثبت‌شده در فراداده همه نتایج
base_url	http://127.0.0.1:4097	نشانی سرور محلی بدون پراکسی که پیکربندی هر آزمایش ثبت کرده است
transport	OpenCode session API	انتخاب خودکار پروتکل؛ نشانی دارای پسوند v1 / سرویس سازگار با OpenAI شمرده می‌شود
provider_id	opencode	شناسه ارائه‌دهنده در دروازه؛ تغییر آن بدون دست‌زدن به کد ممکن است
temperature	0.2	در دمای ۱/۰ یک بسته در دو اجرا نخست تأیید و سپس رد شد
reasoning_effort	low	بزرگ‌ترین اهرم هزینه؛ یک اجرای ۲۰ نمونه‌ای از حدود ۶۴۶ هزار به ۳۶ هزار توکن خروجی رسید
max_tokens	8192 [تا سقف] 32768	بودجه اولیه خروجی و پله‌های دوبرابر شدن آن تا سقف، برای جلوگیری از پاسخ بریده‌شده
timeout	300 [ثانیه]	سقف انتظار هر فراخوانی؛ فراتر از آن به‌عنوان خطای نمونه شمرده می‌شود
تلاش مجدد	2 / 4 / 2	بودجه جداگانه برای شکست کلی، خرابی انتقال و زمان‌متوقف
هم‌زمانی فاز دو	8	شمار فراخوانی‌های هم‌زمان در پیکربندی‌های C و D
سقف گروه بازیابی	50	بیشینه گروه بازیابی برای هر فایل، برای مهار تعداد درخواست‌ها
محیط اجرا	Python 3.14.7	نسخه محیط اجرا، ثبت‌شده در فراداده

جدول ۳.۵: پیکربندی مدل زبانی و اتصال سرویس داوری

مدل به‌کاررفته در همهٔ آزمایش‌ها mimo-v2.6-flash-free است. این شناسه در فیلد model فرادادهٔ هر چهار نتیجه ثبت شده و در پیکربندی B علاوه بر آن، فیلد models_in_file فهرست همهٔ مدل‌های حاضر در پروندهٔ پیش‌بینی‌ها را نگه می‌دارد تا اگر در میان نمونه‌ها مدلی عوض شده باشد، ارزیابی بی‌سروصدا با دو مدل مخلوط ناهمگن انجام نشود. نشانی ثبت‌شده در این اجراها <http://127.0.0.1:4097> است، یعنی یک سرور محلی بدون پراکسی. مدل از مسیر سطح رایگان همان دروازه در دسترس بود: در پیکربندی محیطی این پژوهش هیچ کلید API تنظیم نشده بود و شناسهٔ مدل نیز با پسوند -free می‌آید؛ در نتیجه نه کلیدی در پیکربندی پژوهش ذخیره شد و نه چیزی از آن در گزارش‌ها یا مخزن ثبت گردید. این انتخاب عمدی بود: سرویس محلی در برابر تغییر نسخه، محدودیت نرخ و تغییرات سمت سرویس‌دهنده در میانهٔ یک اجرا پایدارتر است و تکرار آزمایش را بدون هزینهٔ مالی ممکن می‌کند.

انتخاب پروتکل ارتباطی بر پایهٔ نشانی پایه، به‌صورت خودکار انجام می‌شود. اگر مسیر نشانی با v1 / خاتمه یابد یا از پیش به نقطهٔ پایانی chat/completions اشاره کند، سامانه سرویس را سازگار با OpenAI فرض می‌کند و درخواست را به‌صورت POST روی همان نقطهٔ پایانی می‌فرستد؛ در غیر این صورت سرور OpenCode فرض می‌شود و از راه session API با آن سخن می‌گوید. بنابراین این سامانه به یک ارائه‌دهندهٔ خاص وابسته نیست: تنها با تغییر یک متغیر محیطی، همان پشتهٔ کد می‌تواند به یک نقطهٔ پایانی سازگار با OpenAI در اینترنت یا به یک سرور محلی دیگر متصل شود، بدون آنکه حتی یک خط از منطق دآوری تغییر کند. بی‌طرفی در برابر ارائه‌دهنده، هم امکان تکرار آزمایش روی یک نقطهٔ پایانی ثابت را می‌دهد و هم اجازه می‌دهد همین ارزیابی بعدها روی مدل یا ارائه‌دهندهٔ دیگر، بدون بازنویسی سامانه، دوباره اجرا شود.

سه پارامتر جدول ۳.۵ بیش از بقیه بر نتیجه اثر می‌گذارند. نخست دما: با دمای ۱/۰، یک بستهٔ واحد در دو اجرای پیاپی نخست تأیید و سپس رد شد؛ چنین نویزی دقیقاً همان چیزی است که یک ارزیابی نمی‌تواند به هیچ عاملی نسبت دهد. مقدار ۰/۲ اندکی تنوع برای عامل جستجوگر باقی می‌گذارد، اما به آن اجازه نمی‌دهد که خود دآوری را رقم بزند.

دوم، تلاش پنهان استدلال است. در یک مدل استدلالی، بخش بزرگی از توکن‌های خروجی پیش از پاسخ نهایی صرف اندیشیدن می‌شود: اندازه‌گیری انجام‌شده نشان داد که هر رأی با حدود ۱۵۰ توکن استدلال پنهان، پشت پاسخی در حدود ۵۰ توکن تولید می‌شود. تنظیم این سقف با reasoning_effort بزرگ‌ترین اهرم کنترل هزینه است: با مقدار low یک اجرای بیست‌نمونه‌ای از حدود ۶۴۶ هزار توکن خروجی به حدود ۳۶ هزار توکن رسید، بدون آنکه نرخ هشدار نادرست بدتر شود. مقدار minimal اما ارزان‌ترین حالت ممکن نبود: در آن حالت، هشدارهای نادرست پیکربندی «فقط عامل زبانی» دو برابر شد، در حالی که low با همان هزینه، تنها حالتی بود که در پیکربندی‌های ایستا-و-عامل به هشدار نادرست صفر رسید. خالی گذاشتن این فیلد نیز ممکن است؛ در آن حالت فیلد اصلاً ارسال نمی‌شود و خود سرویس دربارهٔ میزان استدلال تصمیم می‌گیرد.

سوم، بودجه توکن خروجی. پارامتر `max_tokens` یک نقطه شروع است، نه سقف نهایی: اگر مدل کل بودجه را صرف استدلال کند و پاسخ بریده شود، سامانه همان درخواست را با بودجه دوباره شده دوباره می‌فرستد تا آن نمونه به خطا تبدیل نشود. این پله‌ها تا سقف تعیین شده بالا می‌روند و پس از آن متوقف می‌شوند. هر تکرار اما یک فراخوانی دیگر روی مسیر بحرانی است، به همین دلیل نقطه شروع بالاتر انتخاب شده تا شمار تکرارها کم بماند و پاسخ‌های موفق کوتاه، هزینه‌ای بیش از حد معمول تحمیل نکنند.

خرابی‌های شبکه نیز بودجه جداگانه خود را دارند: یک بودجه برای شکست کلی، یکی برای خرابی انتقال و یکی برای زمان متوقف؛ و سقف انتظار هر فراخوانی نیز در جدول آمده است. این تفکیک از آن مهم است که قطعی موقت شبکه نباید به عنوان خطای روش‌شناختی سامانه شمرده شود. هم‌زمانی هشت‌گانه فاز دو و سقف پنجاه گروه برای هر فایل نیز مصالحه‌ای میان زمان اجرا و فشار روی سرویس‌اند: نخستی زمان پاسخ‌گویی را کوتاه می‌کند و دومی مانع از آن می‌شود که یک فایل پر از هشدار، تعداد درخواست‌ها را به صورت نامتعارفی بالا ببرد.

سرانجام، همه این مقادیر در کنار شمارش دقیق تصمیم‌ها، یعنی تعداد رد شده‌ها و تأییدها و نامطمئن‌ها، در فراداده نتیجه ثبت می‌شوند. این ثبت، مرز ادعای این پژوهش را نیز روشن می‌کند: اعداد فصل چهارم، توان ترکیب این معماری با این مدل و این پی‌کربندی‌اند، نه خاصیتی ثابت و ماندگار از خود مدل زبانی. انتقال به مدل یا ارائه‌دهنده دیگر نه تنها ممکن، بلکه توصیه‌شدنی است، اما نتیجه آن باید به عنوان یک ارزیابی جداگانه گزارش شود، نه تأییدی بر همین اعداد؛ این موضوع در محدودیت‌های فصل پنجم نیز بازتاب یافته است.

فصل چهارم

ارزیابی تجربی

۴.۱ پرسش‌های پژوهش

ارزیابی تجربی این پژوهش بر پنج پرسش استوار است:

1. بازبینی مبتنی بر شواهد، هشدارهای نادرست تحلیل ایستا را کاهش می‌دهد؟
2. بهای این کاهش از نظر یادآوری چقدر است؟
3. افزودن شواهد ساختاری، اثر بازبینی را بهبود می‌دهد؟
4. ترکیب تحلیل ایستا و عامل زبانی، چه تصویری از منحنی مبادله دقت و یادآوری به دست می‌دهد؟
5. موتور قطعی ارزیابی مسیر، بدون اتکا به مدل زبانی، چه دقت، یادآوری و نرخ هشدار دارد؟

۴.۲ مجموعه داده و آماده‌سازی آن

برای آنکه هر چهار پیکربندی روی یک جمعیت یکسان سنجیده شوند، از مجموعه داده جولیت یک زیرمجموعه متعادل و لایه‌بندی شده ساخته شد: ۳۰۰ فایل آسیب‌پذیر و ۳۰۰ فایل سالم. لایه‌بندی بر پایه نوع آسیب‌پذیری و برچسب انجام شده است، به گونه‌ای که از هر یک از چهار لایه، دقیقاً ۱۵۰ نمونه با بذر ثابت انتخاب شود. زبان و نوع آسیب‌پذیری در این زیرمجموعه درهم‌تنیده‌اند: سرریز عدد صحیح عمدتاً در فایل‌های C و سرریز بافر عمدتاً در فایل‌های C++ متمرکز است. این نکته در تفسیر بخش ۴.۱۴ باید در نظر گرفته شود.

ویژگی	مقدار	توضیح
زبان‌ها	C, C++	هر دو خانواده کد مدل می‌شوند
آسیب‌پذیری‌ها	CWE-122, CWE-190	سرریز بافر و سرریز عدد صحیح
تعداد نمونه ارزیابی	۶۰۰	۳۰۰ آسیب‌پذیر و ۳۰۰ سالم
لایه‌بندی	۱۵۰ در هر لایه	چهار لایه: CWE-122 آسیب‌پذیر، CWE-122 سالم؛ CWE-190 آسیب‌پذیر، CWE-190 سالم؛ بذر ثابت ۴۲
ابزارهای فاز یک	cppcheck 2.22.0, flawfinder 2.0.20	نسخه‌های به‌کاررفته
مدل بازبین	mimo-v2.6-flash-free	از راه سرور محلی، بخش ۳.۱۰

جدول ۴.۱: مشخصات مجموعه داده ارزیابی و پیکربندی آزمایش

برای پرسش پنجم، دو مجموعه مستقل به کار رفت. نخست، یک محک مصنوعی جفتی پایتون شامل ۴۸ فایل: ۲۴ سناریوی آسیب‌پذیر و نسخه اصلاح‌شده همان سناریو در شش دسته دامنۀ پژوهش. دوم، نسخه به‌کاررفته مجموعه دیویگن شامل ۲۷۰۲۵۸ تابع واقعی C.

۴.۳ طراحی آزمایش‌ها

چهار پیکربندی روی زیرمجموعه ۶۰۰ نمونه‌ای اجرا شد:

- **A: فقط ایستا:** تنها فاز یک؛ همه یافته‌های ابزارها بدون فیلتر به‌عنوان خروجی. خط پایه خام.
- **B: فقط عامل زبانی:** فاز یک حذف می‌شود؛ کل فایل در اختیار عامل قرار می‌گیرد. سقف توان عامل زبانی بدون کمک ابزار.
- **C: ایستا + عامل:** یافته‌های ابزارها به‌عنوان نامزد به عامل داده می‌شوند، بدون شواهد ساختاری.
- **D: ایستا + ساختار + عامل:** مانند C، با افزودن مسیرهای منبع تا مصرف. تنها تفاوت با C، وجود یا نبود شواهد ساختاری است.

سه آزمایش بدون مدل زبانی نیز توان موتور قطعی را می‌سنجند (بخش‌های ۴.۹ و ۴.۱۰): همان موتور روی زیرمجموعه ۶۰۰ نمونه‌ای جولیت در برابر فلاپایندر؛ روی محک ۴۸ نمونه‌ای پایتون در برابر بندیت؛ و روی تابع‌های دیویگن در برابر فلاپایندر روی زیرمجموعه متعادل.

دو نکته روش‌شناختی:

- در پیکربندی‌های C، B و D، داوری به‌صورت **تک‌عامله** انجام می‌شود، یعنی از یک فراخوانی واحد عامل زبانی برای هر تصمیم استفاده می‌شود، نه از معماری دوعامله‌ای (جستجوگر + ارزیاب) که در فصل سوم توصیف شد. علت این انتخاب آن است که هدف این فصل، سنجش **اثر خالص شواهد و بازیابی** است، نه سنجش اثر تفکیک نقش‌ها. پیکربندی دوعامله‌ای به‌عنوان کار آینده در بخش ۵.۳ پیشنهاد شده است. این تصمیم یک محدودیت مهم است که در بخش ۴.۱۶ بازتاب یافته.
- دو پیکربندی C و D از هر نظر جز در وجود یا نبود شواهد ساختاری معادل‌اند؛ بنابراین اختلاف آن‌ها تخمین اثر خالص شواهد ساختاری است. زیرمجموعه، ترتیب نمونه‌ها، پاک‌سازی نشت برچسب (فصل ۳، بخش ۳.۶)، بسته شواهد، سیاست تطبیق، آستانه‌های گزارش و مدل زبانی در همه پیکربندی‌های مبتنی بر مدل یکسان نگه داشته شده‌اند.

۴.۴ معیارها و سیاست تطبیق

سه معیار در دانه‌بندی یافته محاسبه می‌شوند: دقت، یادآوری و F1. یک معیار در دانه‌بندی فایل محاسبه می‌شود: نرخ هشدار روی فایل سالم، یعنی سهم فایل‌های سالمی که دست‌کم یک هشدار دریافت کرده‌اند. این تفکیک از آن جهت مهم است که توسعه‌دهنده در عمل با فایلی روبه‌رو می‌شود که هشدار روی آن نشسته است، نه با یک خط منفرد.

معیارها بر پایه سیاست تطبیق فصل ۳ (بخش ۳.۷) محاسبه می‌شوند: یکسان‌بودن فایل، هم‌پوشانی خانواده CWE، و تحمل خط کمتر از ۵ خط. فرمول‌ها:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN},$$

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad \text{FPR} = \frac{FP_{\text{file}}}{FP_{\text{file}} + TN_{\text{file}}}$$

بازه اطمینان ویلسون با سطح ۹۵٪ برای معیارهای اصلی در بخش ۴.۱۲ گزارش می‌شود. تفسیر هر عدد این فصل، بدون دیدن بازه آن، ناقص است.

۴.۵ نتایج اصلی

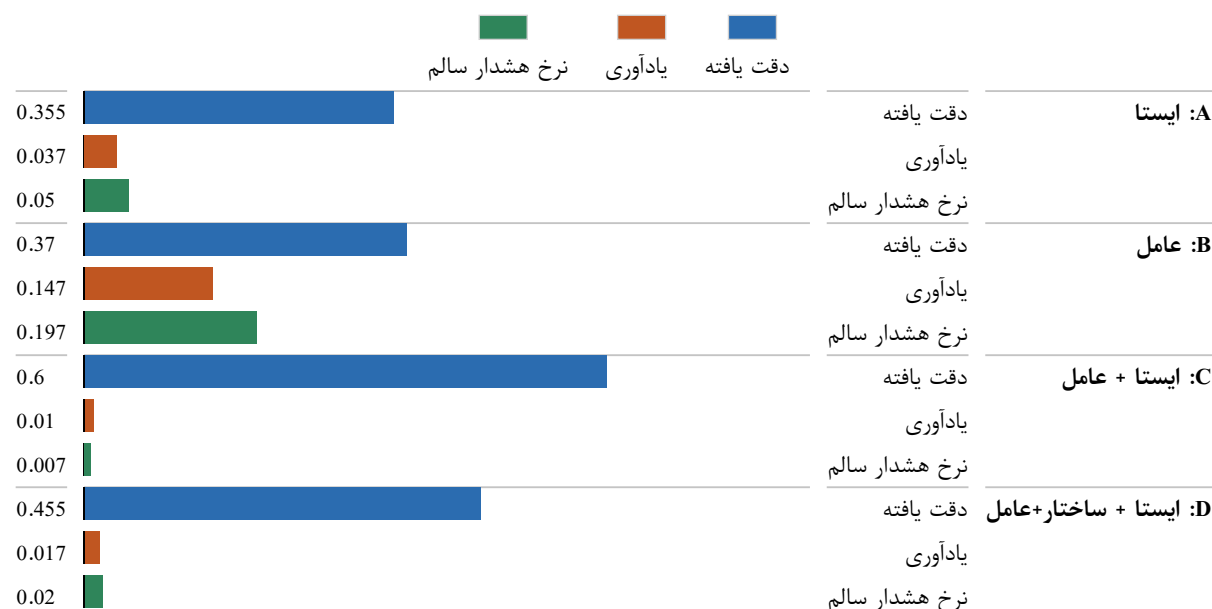
نتایج چهار پیکربندی در جدول ۴.۲ آمده است.

آزمایش	پیکربندی	مثبت درست	مثبت نادرست	دقت	یادآوری	F1	نرخ هشدار روی فایل سالم
A	فقط ایستا	۱۱	۲۰	۰٫۳۵۵	۰٫۳۷	۰٫۰۶۶	۰٫۰۵۰
B	فقط عامل زبانی	۴۴	۷۵	۰٫۳۷۰	۰٫۱۴۷	۰٫۲۱۰	۰٫۱۹۷
C	ایستا + عامل	۳	۲	۰٫۶۰۰	۰٫۱۰	۰٫۰۲۰	۰٫۰۰۷
D	ایستا + ساختار + عامل	۵	۶	۰٫۴۵۵	۰٫۱۷	۰٫۰۳۲	۰٫۰۲۰

جدول ۴.۲: نتایج اصلی چهار آزمایش روی ۶۰۰ نمونه متعادل. «مثبت درست/نادرست» و «دقت» در دانه‌بندی یافته و «نرخ هشدار روی فایل سالم» در دانه‌بندی فایل محاسبه می‌شوند. سبز: بهترین، قرمز: بدترین مقدار هر ستون.

پیکربندی A، یعنی تحلیل ایستا به‌تنهایی، کمترین نرخ هشدار روی فایل سالم را با ۰٫۰۵۰ دارد، اما تنها ۰٫۳۷ از فایل‌های آسیب‌پذیر را کشف می‌کند. این یادآوری پایین، سقف تحلیل الگومحور را نشان می‌دهد.

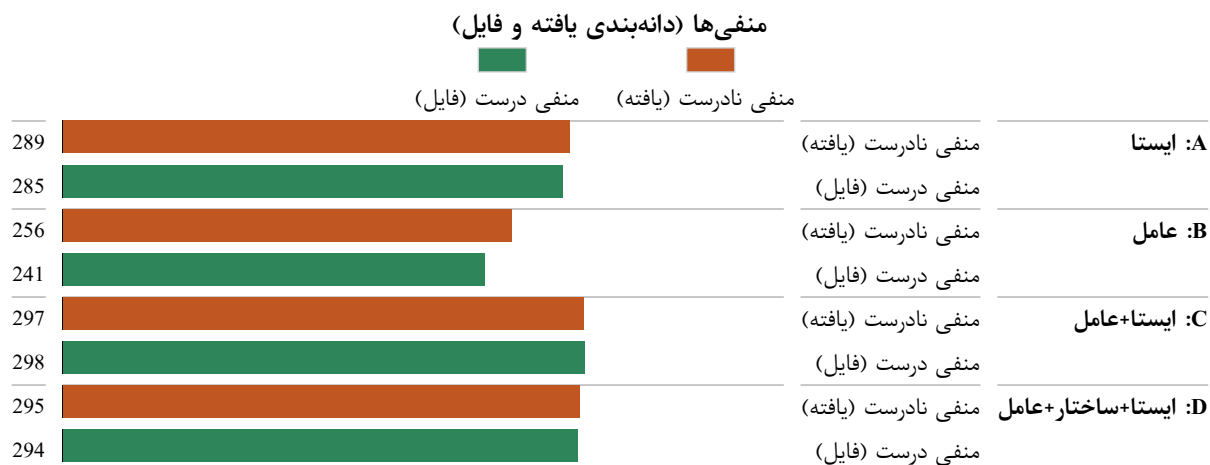
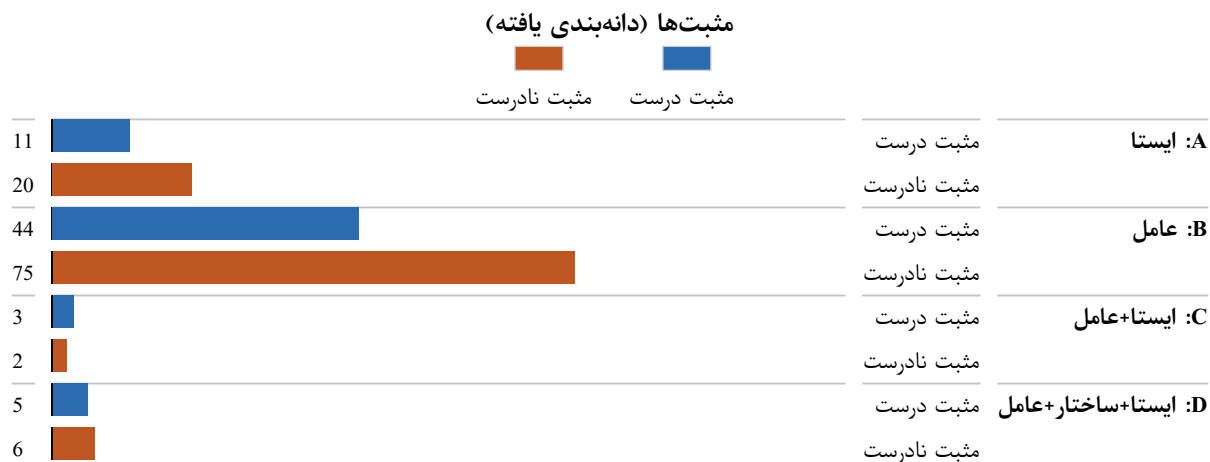
پیکربندی B، یعنی عامل زبانی به‌تنهایی، بالاترین یادآوری را با 0.147 به دست می‌آورد — چهار برابر تحلیل ایستا. این یعنی عامل زبانی چیزهایی می‌بیند که ابزارهای الگومحور از آن‌ها غافل‌اند. بهای این افزایش، بالا رفتن نرخ هشدار روی فایل‌های سالم به 0.197 است؛ یعنی تقریباً یک فایل سالم از هر پنج فایل، هشدار می‌گیرد. دقت یافته‌ها (0.370) تقریباً با پیکربندی A (0.355) برابر است، پس افزایش یادآوری با کاهش کیفیت یافته‌ها همراه نبوده است.



شکل ۴.۱: مقایسه دقت، یادآوری و نرخ هشدار روی فایل سالم در چهار پیکربندی

پاسخ پرسش یکم و دوم. بازبینی مبتنی بر شواهد، نرخ هشدار روی فایل سالم را از 0.050 در A به 0.007 در C رساند؛ یعنی حدود **۸۶ درصد** کاهش. دقت یافته‌ها نیز از 0.355 به 0.600 افزایش یافت. اما بهای این کاهش، افت یادآوری از 0.147 به 0.010 است؛ از یازده یافته درست در A، تنها سه مورد در C باقی ماند. به بیان دیگر، عامل زبانی علاوه بر نادیده گرفتن هشدارهای نادرست، هشت مورد از یافته‌های درست ابزارها را نیز کنار گذاشت.

شکل ۴.۲ ماتریس سردرگمی هر چهار پیکربندی را در دو دانه‌بندی یافته و فایل نشان می‌دهد.



شکل ۴.۲: ماتریس سردرگمی چهار پیکربندی در دانه‌بندی یافته و دانه‌بندی فایل

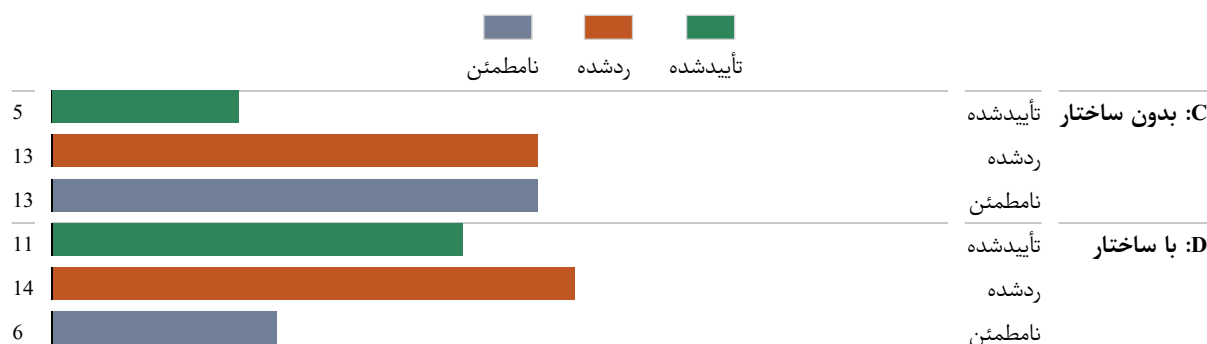
۴.۶ تحلیل قیف تصمیم

قیف تصمیم مسیر فروگاهی است که هر نامزد از لحظه ورود به فاز داوری تا خروجی نهایی طی می‌کند: از نامزد خام، به داوری عامل، و در نهایت به سه سرنوشت ممکن — تأیید، رد، یا نامطمئن. برای فهمیدن دلیل افت یادآوری، باید به این تصمیم‌ها نگاه کرد، نه فقط به معیارهای تجمیعی.

آزمایش	ورودی (گروه/فایل)	تأییدشده	ردشده	نامطمئن
B	۶۰۰ فایل	۱۱۹	۴۷۷	۴
C	۳۱ گروه	۵	۱۳	۱۳
D	۳۱ گروه	۱۱	۱۴	۶

جدول ۴.۳: قیف تصمیم عامل زبانی. برای B واحد شمارش فایل است (عامل بدون فاز یک روی هر فایل داوری می‌کند)، برای C و D واحد شمارش گروه بازبینی است.

در پیکربندی C، عامل از ۳۱ گروه، ۵ گروه را تأیید کرد، ۱۳ گروه را رد کرد و ۱۳ گروه (حدود ۴۲ درصد) با حالت نامطمئن بسته شد. سهم بالای خروجی نامطمئن از دو جهت مهم است: از یک سو، عامل به جای داوری قطعی، کمبود شاهد را صادقانه اعلام می‌کند. از سوی دیگر، چون در خروجی نهایی تنها یافته‌های تأییدشده شمرده می‌شوند، موارد نامطمئن به‌طور خودکار از دست می‌روند.



شکل ۴.۴: اثر شواهد ساختاری بر تصمیم‌های عامل زبانی؛ با افزودن شواهد، تأییدها از ۵ به ۱۱ رسید و نامطمئن‌ها از ۱۳ به ۶ کاهش یافت.

اثر شواهد ساختاری (مقایسه C و D). با افزودن شواهد ساختاری، تعداد تأییدها از ۵ به ۱۱ رسید و موارد نامطمئن از ۱۳ به ۶ کاهش یافت. مثبت‌های درست از ۳ به ۵ افزایش یافت و یادآوری از ۰/۰۱۰ به ۰/۰۱۷ رسید. پاسخ پرسش سوم مثبت است. بهای این افزایش، رشد مثبت‌های نادرست از ۲ به ۶ و رشد نرخ هشدار روی فایل سالم از ۰/۰۰۷ به ۰/۰۲۰ است. شواهد بیشتر، عامل را مطمئن‌تر می‌کند و اطمینان بیشتر، هم یافته درست و هم یافته نادرست بیشتری تولید می‌کند.

۴.۷ نتایج به تفکیک نوع آسیب‌پذیری

آزمایش	نوع	مثبت درست	مثبت نادرست	دقت	یادآوری	F1
A	CWE-122	۱۱	۱۶	۰/۴۰۷	۰/۰۷۳	۰/۱۲۴
A	CWE-190	۰	۴	۰/۰۰۰	۰/۰۰۰	۰/۰۰۰
B	CWE-122	۳۵	۳۳	۰/۵۱۵	۰/۲۳۳	۰/۳۲۱
B	CWE-190	۹	۴۲	۰/۱۷۶	۰/۰۶۰	۰/۰۹۰
C	CWE-122	۳	۲	۰/۶۰۰	۰/۰۲۰	۰/۰۳۹
C	CWE-190	۰	۰	۰/۰۰۰	۰/۰۰۰	۰/۰۰۰
D	CWE-122	۵	۶	۰/۴۵۵	۰/۰۳۳	۰/۰۶۲
D	CWE-190	۰	۰	۰/۰۰۰	۰/۰۰۰	۰/۰۰۰

جدول ۴.۴: نتایج به تفکیک نوع آسیب‌پذیری. مجموع مثبت‌های نادرست در هر پیکربندی با جدول ۴.۲ یکسان است.

هیچ‌یک از سه پیکربندی A، C و D حتی یک نمونه از ۱۵۰ نمونه سرریز عدد صحیح (CWE-190) را کشف نکردند؛ یادآوری این نوع در همه آن‌ها صفر است. تنها پیکربندی B توانست در این نوع چند نمونه را کشف کند (۹ از ۱۵۰). این تفاوت ریشه در ماهیت دو نوع آسیب‌پذیری دارد: سرریز بافر الگوی نحوی شناخته‌شده‌ای دارد و قواعد الگویی ابزارها آن را فعال می‌کنند، در حالی که سرریز عدد صحیح خود یک شکست در مدل نوع است و قاعده‌ای که آن را بگیرد بسیار دشوارتر است.

نتیجه عملی: ابزارهای ایستا برای پوشش این دسته به‌تنهایی کافی نیستند، و سامانه‌های ترکیبی که فایل را به‌طور کامل به مدل زبانی می‌سپارند، پوشش به‌مراتب گسترده‌تری به دست می‌آورند، به بهای نرخ بالاتر هشدار روی فایل‌های سالم.

۴.۸ بررسی در مقیاس کامل

برای بررسی وابستگی نتایج فاز یک به اندازه مجموعه، پیکربندی A علاوه بر زیرمجموعه ۶۰۰ نمونه‌ای، روی کل ۴۰۹۸ فایل مجموعه‌داده نیز اجرا شد. نتیجه با زیرمجموعه هم‌خوان است: دقت ۰/۰۴۵، یادآوری ۰/۰۴۶ و نرخ هشدار روی فایل سالم ۰/۰۵۱. بنابراین محدودیت یادآوری تحلیل ایستا، ویژگی ابزارها و مجموعه‌داده است، نه اثر تصادفی نمونه‌گیری.

۴.۹ ارزیابی موتور شواهد در پایتون

منظور از **موتور شواهد**، مؤلفه قطعی و بدون مدل زبانی سامانه است که با تجزیه درخت نحوی و پیمایش مسیر منبع تا نقطه مصرف، زنجیره‌ای قابل حسابرسی از شواهد جریان داده می‌سازد. این موتور در نسخه فعلی **درون‌تابعی** است؛ یعنی مسیر را فقط در محدوده همان تابع رهگیری می‌کند و ارتباط بین تابع‌ها را نمی‌بیند. این محدودیت، سقف یادآوری را در داده‌های واقعی تعیین می‌کند و در تفسیر اعداد این بخش و بخش ۴.۱۰ باید در نظر گرفته شود.

در این بخش، توان موتور روی ۴۸ فایل پایتون سنجیده می‌شود: ۲۴ سناریوی آسیب‌پذیر و نسخه اصلاح‌شده همان سناریو در شش دسته دامنۀ پژوهش. در این آزمایش هیچ فراخوانی مدل زبانی انجام نمی‌شود؛ پس عددها توان **شواهد** را می‌سنجند، نه توان داوری چندعامله را.

سناریوی ارزیابی	TP / FP	FN / TN	دقت	یادآوری	نرخ هشدار سالم
موتور: همه مسیرها	24/9	0/15	0.727	1.000	0.375
موتور: با فیلتر کاهش خطر	24/1	0/23	0.960	1.000	0.042
بندیت	16/6	8/18	0.727	0.667	0.250
اجتماع موتور (با فیلتر) و بندیت	24/7	0/17	0.774	1.000	0.292

جدول ۴.۵: نتایج موتور قطعی روی محک ۴۸ نمونه‌ای پایتون. ردیف «اجتماع» ترکیب موتور با فیلتر کاهش خطر و بندیت است، نه ترکیب موتور با همه مسیرها. سبز: بهترین، قرمز: بدترین مقدار هر ستون.

اگر همه مسیرهای بازیابی‌شده شمرده شوند، یادآوری کامل است، اما نرخ هشدار به ۰/۳۷۵ می‌رسد. با اعمال قانون کاهش خطر، دقت به ۰/۹۶ و نرخ هشدار به ۰/۰۴۲ می‌رسد و یادآوری همچنان کامل می‌ماند. بندیت در همین مجموعه دقت ۰/۷۲۷، یادآوری ۰/۶۶۷ و نرخ هشدار ۰/۲۵۰ دارد. پس موتور شواهد در این محک کنترل‌شده، هم یادآوری بیشتر و هم هشدار کمتری نسبت به ابزار پایه دارد.

این مقایسه معتبر است، اما دامنۀ آن محدود است: سناریوها مصنوعی‌اند و همراه با موتور نوشته شده‌اند. پس جدول ۴.۵ آزمون بازگشت است، نه ارزیابی مستقل.

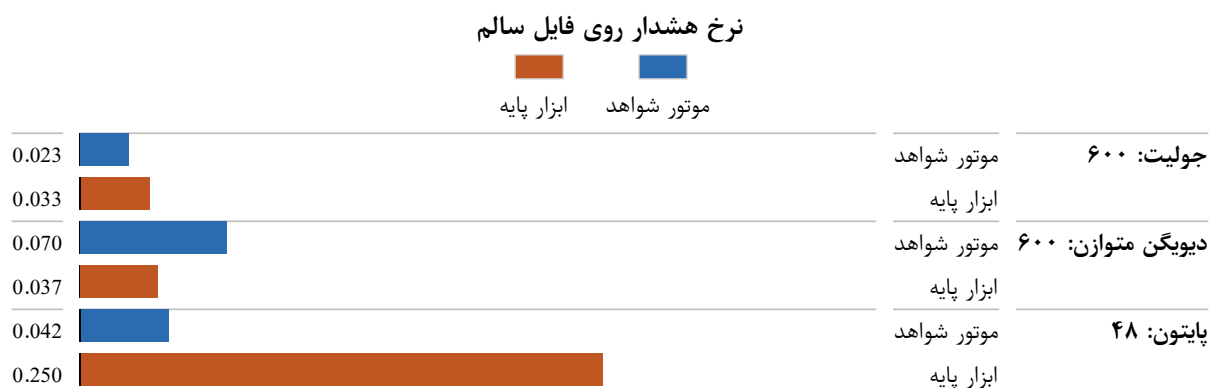
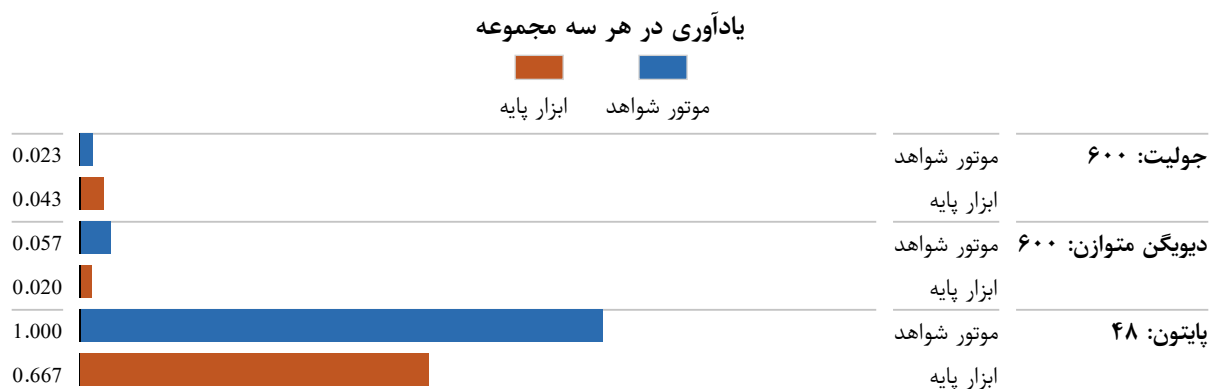
۴.۱۰ ارزیابی موتور شواهد در جولیت و دیویگن

نخست، همان موتور روی زیرمجموعه ۶۰۰ نمونه‌ای جولیت سنجیده می‌شود: ۲۹۹ فایل C و ۳۰۱ فایل C++. دوم، روی ۲۷۲۵۸ تابع واقعی C از مجموعه دیویگن. چون دیویگن برچسب CWE ندارد، تنها دقت، یادآوری و نرخ هشدار کلی گزارش می‌شود.

مجموعه و سناریو	TP / FP	FN / TN	دقت	یادآوری	نرخ هشدار سالم
جولیت: همه مسیرها	26/33	274/267	0.441	0.087	0.110
جولیت: با فیلتر کاهش خطر	7/7	293/293	0.500	0.023	0.023
جولیت: فلافایندر	13/10	287/290	0.565	0.043	0.033
جولیت: اجتماع	13/11	287/289	0.542	0.043	0.037
دیویگن کامل: همه مسیرها	971/894	11454/13939	0.521	0.078	0.060
دیویگن کامل: با فیلتر کاهش خطر	861/783	11564/14050	0.524	0.069	0.053
دیویگن متوازن: موتور (با فیلتر)	17/21	283/279	0.447	0.057	0.070
دیویگن متوازن: فلافایندر	6/11	294/289	0.353	0.020	0.037
دیویگن متوازن: اجتماع	20/29	280/271	0.408	0.067	0.097

جدول ۴.۶: نتایج موتور قطعی روی جولیت و دیویگن

یادآوری پایین در این دو مجموعه، شکست موتور نیست، بلکه پیامد **دامنه درون‌تابعی** آن است. بیشتر موارد جولیت ناهماهنگی اندازه و تخصیص‌اند، نه جریان داده؛ و برچسب دیویگن به‌معنای دست‌کاری تابع در یک وصله امنیتی است، نه لزوماً وجود یک مسیر منبع تا مصرف. به همین دلیل، عدد معنادار این بخش **دقت و نرخ هشدار** در کنار پایه ابزار است، نه یادآوری مطلق.



شکل ۴.۵: توان موتور شواهد در برابر ابزار پایه روی هر سه مجموعه؛ موتور شواهد روی دیویگن یادآوری بیشتر و روی پایتون هم دقت بیشتر و هم هشدار کمتری دارد، اما در جولیت از فلافایندر عقب می‌ماند. عدد یادآوری پایتون ۱/۰۰۰ است.

مجموعه	شاخص	مقدار
جولیت (۶۰۰ فایل)	یادآوری سرریز بافر (CWE-122)	۷ از ۱۵۰، یعنی ۰/۰۴۷
	یادآوری سرریز عدد صحیح (CWE-190)	۰ از ۱۵۰، یعنی ۰/۰۰۰
	یادآوری در زبان C	۳ از ۱۵۸، یعنی ۰/۰۱۹
	یادآوری در زبان C++	۵ از ۱۴۲، یعنی ۰/۰۳۵
	مسیرهای منبع تا مصرف	۶۱ مسیر، همه از دسته سرریز بافر
	زنجیره‌های دارای قاعده کاهش خطر	۴۷ مورد: ۳۱ طول ثابت، ۱۲ مرزدار، ۴ مقصد اندازه‌دار
دیوینگن (۲۷۲۵۸ تابع)	یادآوری	۸۶۱ از ۱۲۰۴۲۵، یعنی ۰/۰۶۹
	مسیرهای منبع تا مصرف	۳۱۵۷ مسیر: ۲۸۷۳ بافر، ۲۶۲ مسیر، ۱۶ فرمان، ۶ کد
	زنجیره‌های دارای قاعده کاهش خطر	۵۱۶ مورد: ۵۰۱ طول ثابت، ۱۴ مرزدار، ۱ مقصد اندازه‌دار

جدول ۴.۷: تفکیک توان موتور شواهد بر پایه خانواده آسیب‌پذیری، زبان و دسته نقطه خطر. «مرزدار» یعنی کاهش خطر با بررسی مرزی انجام شده، «طول ثابت» یعنی طول مقصد در زمان کامپایل معلوم است، و «مقصد اندازه‌دار» یعنی ظرفیت مقصد از یک متغیر مستقل گرفته می‌شود.

۴.۱۱ ساختار هشدارهای نادرست و تحلیل خطاها

معیارهای تجمیعی نشان می‌دهند چه تعداد خطا رخ داده، اما نمی‌گویند این خطاها چه شکلی‌اند. بیست هشدار نادرست پیکربندی A در جدول ۴.۸ تفکیک شده است.

ابزار	خانواده گزارش شده	شمار	سهم	روی فایل سالم	روی فایل آسیب‌پذیر
flawfinder	CWE-120	۱۱	۵۵٪	۱۱	۰
flawfinder	CWE-120, CWE-20	۳	۱۵٪	۱	۲
cppcheck	CWE-190	۴	۲۰٪	۴	۰
cppcheck	CWE-398	۲	۱۰٪	۱	۱
مجموع	—	۲۰	۱۰۰٪	۱۷	۳

جدول ۴.۸: ساختار هشدارهای نادرست پیکربندی A. ستون «روی فایل سالم» و «روی فایل آسیب‌پذیر» نشان می‌دهد هر ردیف کجا نشسته است. ۱۷ هشدار روی ۱۵ فایل سالم پراکنده شده‌اند (دو فایل هرکدام دو هشدار).

نکاتی در این مورد:

- فلافایندر ۱۴ هشدار نادرست و سی‌پی‌پی‌چک ۶ هشدار داد. اما همه یازده یافته درست همین زیرمجموعه نیز از فلافایندر آمده است. یعنی دقت فلافایندر در این زیرمجموعه ۰/۴۴ (۱۱ از ۲۵) و دقت سی‌پی‌پی‌چک دقیقاً صفر (۰ از ۶) است. سی‌پی‌پی‌چک در این زیرمجموعه هیچ سود خالصی به سامانه نرساند. افزودن ابزار، لزوماً به معنای افزودن سیگنال نیست.
- سه مورد از بیست هشدار روی فایل‌های آسیب‌پذیر نشسته‌اند، اما خانواده CWE آن‌ها با برچسب هم‌پوشانی ندارد (خطای گزارش‌دهی، نه خطای یافتن). هفده مورد دیگر روی نسخه سالم همان سناریو نشسته‌اند: الگویی که در کل فایل وجود دارد، اما در آن نسخه به نقص امنیتی تبدیل نشده است. این هفده هشدار روی ۱۵ فایل سالم پراکنده‌اند (دو فایل هرکدام دو هشدار).
- پیکربندی C هجده مورد از این بیست هشدار را کنار گذاشت و D چهارده مورد. اما هر دو پیکربندی یافته‌های تازه‌ای فراتر از خط پایه تولید نکردند: خروجی آن‌ها زیرمجموعه‌ای از همان مجموعه خط پایه است.
- در A، این دسته صفر یافته درست و چهار هشدار نادرست داشت. پس از بازبینی، C و D نه هشدار در این دسته باقی گذاشتند و نه یافته‌ای. حذف نویز در اینجا دستاورد ارزشمندی است، اما جایگزین کشف آسیب‌پذیری نیست.
- از یازده یافته درست A، پیکربندی C سه مورد و D پنج مورد را حفظ کرد، اما تنها دو مورد در هر دو مشترک‌اند. شواهد ساختاری پاسخ عامل را بازنویسی می‌کند، نه آنکه صرفاً به آن بیفزاید. خالص افزایش یادآوری در D، نتیجه دو جابه‌جایی در دو جهت مخالف است.

۴.۱۲ بازه‌های اطمینان و حساسیت به اندازه نمونه

شمار یافته‌های درست در C و D کوچک است (۳ و ۵) و بدون محاسبه عدم قطعیت، هر تفسیری درباره برتری آن‌ها بیش از حد قاطع است.

معیار	آزمایش	پیکربندی	برآورد نقطه‌ای	بازه اطمینان ۹۵٪
دقت	A	فقط ایستا	۰٫۳۵۵	۰٫۲۱۱ تا ۰٫۵۳۱
دقت	B	فقط عامل زبانی	۰٫۳۷۰	۰٫۲۸۸ تا ۰٫۴۵۹
دقت	C	ایستا + عامل	۰٫۶۰۰	۰٫۲۳۱ تا ۰٫۸۸۲
دقت	D	ایستا + ساختار + عامل	۰٫۴۵۵	۰٫۲۱۳ تا ۰٫۷۲۰
یادآوری	A	فقط ایستا	۰٫۰۳۷	۰٫۰۲۱ تا ۰٫۰۶۴
یادآوری	B	فقط عامل زبانی	۰٫۱۴۷	۰٫۱۱۱ تا ۰٫۱۹۱
یادآوری	C	ایستا + عامل	۰٫۰۱۰	۰٫۰۰۳ تا ۰٫۰۲۹
یادآوری	D	ایستا + ساختار + عامل	۰٫۰۱۷	۰٫۰۰۷ تا ۰٫۰۳۸
نرخ هشدار فایل سالم	A	فقط ایستا	۰٫۰۵۰	۰٫۰۳۱ تا ۰٫۰۸۱
نرخ هشدار فایل سالم	B	فقط عامل زبانی	۰٫۱۹۷	۰٫۱۵۶ تا ۰٫۲۴۵
نرخ هشدار فایل سالم	C	ایستا + عامل	۰٫۰۰۷	۰٫۰۰۲ تا ۰٫۰۲۴
نرخ هشدار فایل سالم	D	ایستا + ساختار + عامل	۰٫۰۲۰	۰٫۰۰۹ تا ۰٫۰۴۳

جدول ۴.۹: برآورد نقطه‌ای و بازه اطمینان ویلسون ۹۵٪ معیارهای اصلی. ترتیب اعداد در هر بازه، از کران پایین به کران بالا.

بازه C از ۰٫۲۳۱ تا ۰٫۸۸۲ و بازه D از ۰٫۲۱۳ تا ۰٫۷۲۰ است. ترتیب ظاهری دقت‌ها ($C > D > B > A$) در سطح ۹۵٪ قابل تفکیک نیست. عبارت «دقت ۶۰۰٪» باید به عنوان «بیشترین برآورد، با بازه ۲۳٪ تا ۸۸٪» خوانده شود، نه به عنوان اثبات برتری.

بازه نرخ هشدار برای B از ۰٫۱۵۶ تا ۰٫۲۴۵ و برای C از ۰٫۰۰۲ تا ۰٫۰۲۴ است؛ این دو بازه اصلاً هم‌پوشانی ندارند. بازه A (۰٫۰۳۱ تا ۰٫۰۸۱) نیز از C جداست. پس ادعای محوری پژوهش «کاهش معنا دار هشدار نادرست» در برابر عدم قطعیت نمونه مقاوم می‌ماند.

بازه B (۰٫۱۱۱ تا ۰٫۱۹۱) از بازه A (۰٫۰۲۱ تا ۰٫۰۶۴) جداست؛ ادعای چهار برابر بودن یادآوری عامل زبانی نسبت به تحلیل ایستا آماری قابل دفاع است. اما بازه‌های C و D با بازه A هم‌پوشانی دارند؛ افت یادآوری ناشی از بازبینی، با این شمار نمونه در سطح ۹۵٪ از صفر متمایز نیست. برای اثبات آماری این افت، به نمونه بزرگ‌تری نیاز است.

۴.۱۳ هزینه، زمان و پایداری اجرا

بازبینی زبانی با تحلیل ایستا در هزینه قابل مقایسه نیست. روی همان ۶۰۰ فایل، اجرای فاز یک (پیکربندی A) ۲۲/۲ ثانیه طول کشید، در حالی که مجموع زمان درخواست‌های ثبت‌شده در پیکربندی B برابر ۹۴۰۹۶۶/۷ ثانیه بود؛ یعنی حدود ۲۶ ساعت زمان درخواست سریالی. اگر این دو عدد بر هم تقسیم شوند، هر فراخوانی مدل زبانی به‌طور میانگین حدود ۴۳۰۰ برابر اجرای کل ابزارهای ایستا روی همان مجموعه هزینه دارد.

این میانگین با اجرای پیکربندی C نیز هم‌خوان است: ۳۱ گروه بازبینی، ۶۳۶/۹ ثانیه زمان ثبت‌شده، و با هم‌زمانی هشت، حدود ۱۶۴ ثانیه به ازای هر تصمیم. هم‌سازی این دو عدد مستقل نشان می‌دهد سهم اصلی زمان، صرف خود استدلال مدل می‌شود، نه آماده‌سازی ورودی یا انتقال.

با این حال، زمان دیواری معیار قابل اتکایی برای مقایسه نیست. پیکربندی D با همان ۳۱ گروه، زمان ۶۷/۴ ثانیه دارد؛ یعنی حدود یک‌دهم C. این اختلاف با حجم کار سامانه توضیح داده نمی‌شود و نشان می‌دهد زمان اجرا تابع وضعیت لحظه‌ای سرویس است. به همین دلیل، زمان اجرا در این پژوهش به‌عنوان معیار مقایسه‌ای گزارش نمی‌شود.

پایداری. در اجرای ۶۰۰ نمونه‌ای پیکربندی B، شمار نمونه‌های خطا دار و ناقص هر دو صفر بود: ۴۷۷ رد، ۱۱۹ تأیید و ۴ نامطمئن. اما روی کل ۴۰۹۸ فایل جولیت، ۱۰۶۸ درخواست در زمان ۱۸۰ ثانیه قطع شد و ۶ پاسخ قابل تجزیه نبود. اعداد این فصل همه از اجرای ۶۰۰ نمونه‌ای بدون خطا آمده‌اند.

۴.۱۴ تفکیک زبانی نتایج

پیکربندی	زبان	کل فایل	آسیب‌پذیر	سالم	TP / FP	یادآوری
A	C	۲۹۹	۱۵۸	۱۴۱	3 / 11	۰/۰۱۹
A	C++	۳۰۱	۱۴۲	۱۵۹	8 / 9	۰/۰۵۶
B	C	۲۹۹	۱۵۸	۱۴۱	19 / 34	۰/۱۲۰
B	C++	۳۰۱	۱۴۲	۱۵۹	25 / 41	۰/۱۷۶
C	C	۲۹۹	۱۵۸	۱۴۱	1 / 1	۰/۰۰۶
C	C++	۳۰۱	۱۴۲	۱۵۹	2 / 1	۰/۰۱۴
D	C	۲۹۹	۱۵۸	۱۴۱	1 / 3	۰/۰۰۶
D	C++	۳۰۱	۱۴۲	۱۵۹	4 / 3	۰/۰۲۸

جدول ۴.۱۰: نتایج چهار پیکربندی به تفکیک زبان. یادآوری نسبت به شمار فایل‌های آسیب‌پذیر همان زبان محاسبه می‌شود. چون زبان و نوع آسیب‌پذیری در این زیرمجموعه درهم‌تنیده‌اند، ستون یادآوری نباید به‌عنوان اثر خالص زبان تفسیر شود (بخش ۴.۱۶).

این نرخ در هر دو زبان تقریباً یکسان است (اختلاف کمتر از ۰/۰۰۵ در همهٔ پیکربندی‌ها). این هم‌سانی تصادفی نیست: عامل تولیدکنندهٔ هشدار نادرست یک الگوی نحوی ساده است — کپی یا الحاق بدون کنترل اندازه — که در هر دو زبان یک شکل دارد. مهار هشدار نادرست وابسته به زبان نیست.

در A، نرخ کشف برای C برابر ۰/۰۱۹ و برای C++ برابر ۰/۰۵۶ است. تفسیر مستقیم این تفاوت گمراه‌کننده است، زیرا زبان و نوع آسیب‌پذیری در هم‌تنیده‌اند: از ۳۰۰ نمونهٔ سرریز عدد صحیح، ۱۹۹ در C و از ۳۰۰ نمونهٔ سرریز بافر، ۲۰۰ در C++ هستند. چون همهٔ یافته‌های درست A از دستهٔ سرریز بافر است، بخش بزرگی از تفاوت ظاهری زبانی بازتاب پراکندگی نامتقارن دو نوع آسیب‌پذیری است. جدا کردن اثر خالص زبان نیازمند نمونه‌گیری مستقل نوع و زبان است؛ چنین زیرمجموعه‌ای در این پژوهش ساخته نشده.

۴.۱۵ تحلیل کیفی دو تصمیم و ارزش امتیاز اعتماد

اعداد تجمیعی نشان می‌دهند چه تعداد تصمیم درست یا نادرست بوده، اما دربارهٔ شکل استدلال پشت هر تصمیم ساکت‌اند. این بخش یک تصمیم درست و یک تصمیم نادرست از پیکربندی D را گام‌به‌گام مقایسه می‌کند.

تصمیم نادرست: فایل نسخهٔ سالم یک سناریوی سرریز بافر، خط ۳۴. فاز یک با قاعدهٔ FF1005 یک فراخوانی `strcat(data, source)` را گزارش می‌کند. بستهٔ شواهد نشان می‌دهد `data` اشاره‌گری است که از آرایهٔ `dataArray[2]` آمده، `source` رشته‌ای ۹۹ بایتی است، و در تابع تنها چهار فراخوانی `memset / strcat / printLine / free` وجود دارد، با صفر شرط کنترل اندازه. ارزیاب یافته را با اطمینان ۰/۹۰ تأیید می‌کند. خود ارزیاب در استدلالش می‌نویسد که ظرفیت بافر مقصد `outside the analyzed function` تعیین می‌شود. یعنی ارزیاب نبود شاهد سلامت را تشخیص داده، اما آن را به‌جای عدم قطعیت، به‌عنوان شاهد آسیب‌پذیری خوانده است.

تصمیم درست: همان سناریو در نسخهٔ آسیب‌پذیر، خط ۳۳. فلاپایندر یک `strep` را گزارش می‌کند و ارزیاب با اطمینان ۰/۹۰ تأیید می‌کند. استدلال: مقصد کپی، بافر پشته‌ای ثابت `char dest[50]` است و رشتهٔ ورودی بدون اعتبارسنجی اندازه به آن ریخته می‌شود؛ همهٔ اطلاعات لازم در همان تابع موجود است.

تفاوت یافتهٔ درست و نادرست نه در قاعدهٔ ابزار است و نه در شکل استدلال، بلکه در این است که **اندازهٔ بافر مقصد کجا تخصیص یافته**. اگر مقصد درون تابع باشد، شواهد کامل‌اند؛ اگر بیرون باشد، شواهد ناقص‌اند و پاسخ درست عدم قطعیت است. ریشهٔ مسئله دو لایه دارد: شعاع هشت‌خطی و موتور درون‌تابعی نمی‌توانند محل تخصیص را در فراخواننده ببینند؛ و داوری بر پایهٔ نبود شاهد منفی انجام شده، نه وجود شاهد مثبت.

در پیکربندی C، اعتماد سه یافتهٔ درست ۰/۷۲، ۰/۸۵، ۰/۹۳ و اعتماد دو یافتهٔ نادرست ۰/۸۶ و ۰/۹۳ بود؛ دو بازه کاملاً روی هم می‌افتند. در پیکربندی D وضعیت حتی برعکس است: میانگین اعتماد پنج یافتهٔ درست ۰/۸۱ و میانگین اعتماد شش یافتهٔ نادرست ۰/۹۲ است. این بدان معنا نیست که امتیاز اعتماد بی‌ارزش است، بلکه بدان معناست که

این امتیاز، **اعتماد خود مدل به استدلالش** را می‌سنجد، نه درستی آن را. کاربرد درست این امتیاز، رتبه‌بندی موارد نامطمئن برای بررسی انسانی است، نه جایگزینی برای سنجش بیرونی.

۴.۱۶ تهدیدهای روایی

در تفسیر نتایج باید به هشت محدودیت اصلی توجه کرد:

1. **اندازه نمونه:** شمار یافته‌های درست در C و D کوچک است (۳ و ۵). با این شمار، هر تفاوت کوچکی نرخ‌ها را به شدت جابه‌جا می‌کند و بازه‌های اطمینان پهن می‌شوند.
2. **محدودیت مجموعه داده:** کد جولیت قالب‌محور و کوتاه است؛ محک پایتون مصنوعی است؛ برچسب دیویگن از وصله‌های تاریخی می‌آید و نویز دارد. هیچ‌کدام جایگزین پروژه‌های واقعی صنعتی نیستند.
3. **تک‌مدلی بودن:** همه پیکربندی‌ها روی یک مدل زبانی مشخص ارزیابی شده‌اند. تعمیم نتایج به سایر مدل‌ها نیازمند آزمایش تکمیلی است.
4. **سیاست تطبیق سخاوتمندانه:** چون برچسب دقیق خط در مجموعه داده ثبت نشده، تطبیق بر پایه فایل و خانواده CWE انجام می‌شود. این سیاست تا حدی سخاوتمندانه است.
5. **درون‌تابعی بودن موتور شواهد:** پیوند منبع تا مصرف فقط در محدوده همان تابع رهگیری می‌شود و ارتباط بین تابع‌ها دیده نمی‌شود. این مهم‌ترین عامل پایین بودن یادآوری در داده‌های واقعی است.
6. **ارزیابی نشدن معماری دوعامله:** اگرچه مؤلفه‌های معماری به صورت مهندسی پیاده‌سازی و آزمون شده‌اند، اثر تجربی تفکیک نقش جستجوگر و ارزیاب با مدل زنده سنجیده نشده است. پیکربندی‌های C، B و D در این فصل تک‌عامله‌اند.
7. **درهم‌تنیدگی زبان و نوع:** در زیرمجموعه ارزیابی، سرریز عدد صحیح عمدتاً در C و سرریز بافر عمدتاً در C++ متمرکز است. هیچ تفاوت زبانی گزارش شده نباید به عنوان اثر خالص زبان تفسیر شود.
8. **عدم تفکیک‌پذیری امتیاز اعتماد:** در پیکربندی D، میانگین اعتماد یافته‌های نادرست حتی بالاتر از یافته‌های درست بود. امتیاز اعتماد را نمی‌توان به عنوان فیلتر به کار برد؛ کاربرد آن فقط رتبه‌بندی موارد نامطمئن برای بررسی انسانی است.

عامل زبانی، هشدارهای نادرست تحلیل ایستا را به طور چشمگیری کاهش می‌دهد (۸۶٪ کاهش در نرخ هشدار روی فایل سالم)، اما این کاهش با افت محسوس یادآوری همراه است، و افزودن شواهد ساختاری این مبادله را به نقطه کاری متعادل‌تری می‌رساند. مرز ادعای این پژوهش، کاهش هشدار نادرست و برتری یادآوری عامل زبانی بر تحلیل ایستاست؛ هر ادعای فراتر از آن، نیازمند نمونه بزرگ‌تر، مجموعه داده واقعی‌تر و اندازه‌گیری مستقل اعتماد است.

فصل پنجم

نتیجه‌گیری و کارهای آینده

۵.۱ جمع‌بندی

در این پژوهش، سامانه‌ای سه‌فاز برای ترکیب تحلیل ساختاری کد با استدلال مدل‌های زبانی بزرگ در قالب معماری چندعامله، به‌منظور کاهش هشدارهای نادرست تحلیل ایستا، طراحی، پیاده‌سازی و ارزیابی شد. خروجی اصلی این سامانه، گزارشی است که تنها یافته‌های دارای شاهد کافی را در بر می‌گیرد و در کنار هر یافته، دلیل و شواهد آن را نیز ارائه می‌کند.

مهم‌ترین یافته تجربی این پژوهش آن است که بازبینی مبتنی بر شواهد، نرخ هشدار روی فایل‌های سالم را از ۰/۰۵۰ به ۰/۰۰۷ رساند؛ یعنی حدود ۸۷ درصد کاهش، و دقت یافته‌ها را از ۰/۳۵۵ به ۰/۶۰۰ افزایش داد. این کاهش اما رایگان نبود: یادآوری از ۰/۰۳۷ به ۰/۰۱۰ افت کرد. عامل زبانی، بخشی از یافته‌های درست ابزارهای ایستا را نیز به‌دلیل کمبود شاهد کنار گذاشت.

سه نتیجه دیگر، تصویر کامل‌تری از توان این رویکرد به دست می‌دهند. نخست، تحلیل ایستا به‌تنهایی سقف یادآوری پایینی دارد و این سقف، از توان داوری سامانه مستقل است. دوم، عامل زبانی به‌تنهایی، با یادآوری ۰/۱۴۷، چهار برابر تحلیل ایستا آسیب‌پذیری یافت، اما نرخ هشدار روی فایل سالم آن به ۰/۱۹۷ رسید. تنها پیکربندی‌ای که به هر دو نوع آسیب‌پذیری مجموعه داده پوشش داد، پیکربندی B بود؛ به‌ویژه برای سرریز عدد صحیح که هیچ‌یک از پیکربندی‌های مبتنی بر تحلیل ایستا حتی یک نمونه از ۱۵۰ نمونه را کشف نکردند. سوم، موتور قطعی بازبینی مسیر روی محک کنترل‌شده پایتون، با شرط کاهش خطر، دقت ۰/۹۶ و نرخ هشدار ۰/۰۴۲ را با یادآوری کامل به دست آورد؛ و روی تابع‌های واقعی دیویگن، دقت ۰/۵۲۴ و نرخ هشدار ۰/۰۵۳ را ثبت کرد.

پاسخ پنج پرسش پژوهش، همراه با شاهد تجربی هر یک، در جدول ۵.۱ جمع شده است. خواندن این جدول در کنار هم، مرز دقیق ادعا را نشان می‌دهد: آنچه با اطمینان قابل طرح است، کاهش هشدار نادرست و برتری یادآوری عامل زبانی تنها بر تحلیل ایستاست؛ و آنچه با این اندازه نمونه قابل طرح نیست، رتبه‌بندی قطعی دقت چهار پیکربندی است.

پرسش پژوهش	پاسخ	شاهد تجربی
آیا بازبینی مبتنی بر شواهد هشدارهای نادرست تحلیل ایستا را کاهش می‌دهد؟	بله. نرخ هشدار روی فایل سالم از ۰/۰۵۰ به ۰/۰۰۷ رسید؛ یعنی حدود ۸۷ درصد کاهش، و دقت از ۰/۳۵۵ به ۰/۶۰۰ افزایش یافت.	جدول‌های 4.2 و 4.9
بهای این کاهش از نظر یادآوری چقدر است؟	یادآوری از ۰/۰۳۷ به ۰/۰۱۰ افت کرد؛ از یازده یافته درست خط پایه، تنها سه مورد باقی ماند.	بخش‌های 4.5 و 4.6

پرسش پژوهش	پاسخ	شاهد تجربی
افزودن شواهد ساختاری چه اثری دارد؟	تأییدها از ۵ به ۱۱ و خروجی‌های نامطمئن از ۱۳ به ۶ رسید؛ یادآوری به ۰/۰۱۷ بالا رفت اما نرخ هشدار به ۰/۰۲۰.	جدول 4.3 و شکل 4.4
ترکیب دو مؤلفه چه منحنی‌ای از مبادله می‌دهد؟	هیچ پیکربندی برتری مطلق ندارد: B بیشترین یادآوری (۰/۱۴۷) و سنگین‌ترین هشدار (۰/۱۹۷) را دارد و C کمترین هشدار (۰/۰۰۷) را.	جدول 4.2 و شکل 4.1
موتور قطعی بازیابی مسیر بدون مدل زبانی چه می‌کند؟	روی محک پایتون با شرط کاهش خطر، دقت ۰/۹۶ و نرخ هشدار ۰/۰۴۲ با یادآوری کامل؛ روی توابع دیویگن، دقت ۰/۵۲۴ و نرخ هشدار ۰/۰۵۳.	جدول‌های 4.5 و 4.6

جدول ۵.۱: پاسخ پنج پرسش پژوهش همراه با شاهد تجربی

دستاوردهای این پژوهش را می‌توان در چهار بند خلاصه کرد. نخست، معماری سه‌فازی که تحلیل ایستا، داوری زبانی و ارزیابی مستقل را از یکدیگر جدا می‌کند و به هر فاز اجازه می‌دهد جداگانه سنجیده شود. دوم، بسته شواهد به‌همراه سازوکارهای مهار اعتماد: پاک‌سازی نشت برچسب، مهار قطعی اعتماد ارزیاب، الزام سازگاری ادعای زنجیره با رأی ارزیاب، و رد قطعی یافته در نبود مسیر واقعی. سوم، موتور قطعی بازیابی مسیر که بدون هیچ فراخوانی مدلی، روی محک پایتون با اعمال شرط کاهش خطر به دقت ۰/۹۶ و نرخ هشدار ۰/۰۴۲ با یادآوری کامل رسید. چهارم، ارزیابی سیستماتیک چهار پیکربندی روی یک جمعیت واحد، همراه با تفکیک خطاها، بازه‌های اطمینان و گزارش هزینه؛ به‌گونه‌ای که هر عدد این پایان‌نامه از روی خروجی‌های بایگانی‌شده قابل بازتولید باشد.

یک دستاورد فرعی اما ارزشمند، خود فراداده است. ثبت نام مدل، نشانی سرویس، نسخه ابزارها، خط تحمل، هم‌زمانی و زمان اجرا در کنار هر نتیجه، آنچه را که بسیاری از پژوهش‌های مشابه از قلم می‌اندازند به بخشی جدانشدنی خود نتیجه تبدیل کرده است. همین ثبت بود که امکان اصلاح سیاست شمارش هشدارهای نادرست را، بی‌آنکه حتی یک فراخوانی مدل تکرار شود، فراهم کرد؛ و همان امکان بود که فصل چهارم را با بخش‌های تفکیک خطا، بازه‌های اطمینان و تحلیل هزینه گسترش داد. به این معنا، تکرارپذیری در این پژوهش نه یک ادعای جانبی، بلکه پیش‌شرط هر عددی است که گزارش شده است.

۵.۲ محدودیت‌ها

با وجود دستاوردهای ارائه‌شده، محدودیت‌های این پژوهش را می‌توان در هشت دسته مجزا و اساسی طبقه‌بندی کرد که تأثیر مستقیم بر تفسیر نتایج دارند. نخستین مسئله، محدودیت شدید در اندازه نمونه است؛ تعداد یافته‌های درستی که در پیکربندی‌های ترکیبی ثبت شده بسیار کوچک است و این باعث می‌شود نرخ‌های گزارش‌شده بسیار حساس و شکننده باشند. دومین مسئله به محدودیت خود مجموعه داده باز می‌گردد؛ کد موجود در مجموعه داده جولیت به شدت قالب‌محور و کوتاه است، در نتیجه یافته‌های استخراج‌شده از آن لزوماً به کدهای واقعی و پروژه‌های نرم‌افزاری بزرگ در صنعت تعمیم نمی‌یابد. به همین منوال، محک‌های پایتون نیز کاملاً مصنوعی هستند و برچسب‌های دیویگن از نویز تاریخی رنج می‌برند.

سومین محدودیت متوجه مدل هوش مصنوعی است؛ ارزیابی‌های صورت‌گرفته تنها روی یک مدل زبانی خاص انجام شده و رفتار مدل‌های دیگر می‌تواند متفاوت باشد. چهارمین محدودیت مربوط به سیاست تطبیق سامانه است؛ با توجه به آنکه برچسب دقیقی خط کد در مجموعه داده ثبت نشده است، تطبیق بر پایه فایل و خانواده CWE انجام می‌شود و این سیاست تطبیق، تا حد زیادی سخاوتمندانه محسوب می‌گردد. پنجمین محدودیت ناشی از درون‌تابعی بودن موتور شواهد است؛ متأسفانه پیوند ایجادشده میان منبع و مصرف متغیرها، در میان تابع‌های مختلف رهگیری نمی‌شود که نقطه ضعفی برای ردیابی جامع است. در نهایت، ششمین محدودیت مربوط به اعتبارسنجی معماری چندعامله است که هرچند از نظر مهندسی به‌خوبی پیاده‌سازی و آزمون شده، اما اثر تجربی و واقعی تفکیک جستجوگر و ارزیاب هنوز از طریق اجرای زنده (Live execution) با یک مدل در مقیاس صنعتی سنجیده نشده است.

هفتمین محدودیت، در هم‌تنیدگی زبان و نوع آسیب‌پذیری در زیرمجموعه ارزیابی است؛ سرریز عدد صحیح عمدتاً در فایل‌های C و سرریز بافر عمدتاً در فایل‌های C++ پراکنده است. در نتیجه، هیچ تفاوت زبانی گزارش‌شده در فصل چهارم نباید به‌عنوان اثر خالص زبان تفسیر شود و تنها در صورتی می‌توان چنین تفکیکی ساخت که نوع آسیب‌پذیری و زبان به‌طور مستقل نمونه‌گیری شده باشند. هشتمین محدودیت، اتکای نامطمئن به امتیاز اعتماد عامل است؛ در پیکربندی D میانگین اعتماد یافته‌های نادرست بالاتر از یافته‌های درست بود. این بدان معناست که نمی‌توان بر پایه آستانه اعتماد، خروجی را پالایش کرد، و افزودن چنین فیلتری بدون سنجش بیرونی ممکن است دقیقاً برعکس عمل کند. بنابراین تا زمانی که رابطه میان اعتماد گزارش‌شده و درستی آن اندازه‌گیری نشده باشد، امتیاز اعتماد تنها برای رتبه‌بندی موارد نامطمئن به‌قصد بررسی انسانی کاربرد دارد. جمع این هشت محدودیت یک جمع‌بندی واحد دارد: مرز ادعای این پژوهش، کاهش هشدار نادرست و برتری یادآوری عامل زبانی بر تحلیل ایستاست؛ هر ادعای فراتر از آن، نیازمند نمونه بزرگ‌تر، مجموعه داده واقعی‌تر و اندازه‌گیری مستقل اعتماد است.

۵.۳ کارهای آینده

بر پایه نتایج به‌دست‌آمده و محدودیت‌های شناسایی‌شده در این پژوهش، مسیرهای روشن متعددی برای ادامه کار و ارتقای سامانه پیشنهاد می‌شود:

- **بازطراحی ساختار بسته شواهد:** گام نخست می‌تواند تمرکز بر تغییر محتوای بسته شواهد به‌گونه‌ای باشد که به‌جای ارائه تنها یک قطعه ایزوله از کد، مسیر جریان داده میان منبع و نقطه خطر را به‌طور کامل در اختیار عامل ارزیاب قرار دهد. انتظار می‌رود با این تغییر کلیدی، مهم‌ترین گلوگاه فعلی سامانه، یعنی کاهش هم‌زمان و نامطلوب معیار یادآوری در کنار هشدارهای نادرست، برطرف گردد.
- **استفاده بهینه از خروجی‌های نامطمئن:** به‌جای دور انداختن مواردی که با عدم قطعیت مواجه شده‌اند، می‌توان راهکاری برای رتبه‌بندی آن‌ها اندیشید. از آنجا که بررسی‌ها نشان داد ۴۲ درصد از گروه‌ها در پیکربندی C با حالت نامطمئن بسته شدند، جمع‌آوری، رتبه‌بندی و ارائه آن‌ها در یک داشبورد یا صفحه جداگانه، می‌تواند امکان بازیابی بخشی از یادآوری ازدست‌رفته را، بدون افزودن مستقیم هشدارهای نادرست به خروجی اصلی، فراهم کند.
- **تنظیم‌پذیری سامانه بر پایه هزینه خطا:** توسعه‌دهندگان می‌توانند نقطه کاری سامانه را بر پایه هزینه نسبی هر نوع خطا در محیط‌های مختلف تنظیم کنند. با تعریف آستانه‌های متفاوت برای تأیید عامل زبانی، سامانه قادر خواهد بود برای محیط‌هایی که وجود هشدار نادرست در آن‌ها به‌شدت پرهزینه است، و یا برعکس، برای سازمان‌هایی که از دست رفتن یک آسیب‌پذیری پیامدهای فاجعه‌باری دارد، سفارشی‌سازی شود.
- **توسعه محیط ارزیابی روی پروژه‌های واقعی:** ارزیابی معماری روی مجموعه داده‌های دیگر و در محیط کدهای واقعی یک ضرورت است. تکرار دقیق آزمایش‌ها روی مجموعه داده‌هایی با کدهای بلندتر که دارای وابستگی‌های پیچیده بین‌فایلی هستند، به‌خوبی مشخص می‌کند که نتایج به‌دست‌آمده در این پایان‌نامه تا چه اندازه به ویژگی‌های خاص مجموعه داده جولیت وابسته‌اند و تا چه حد قابل تعمیم به پروژه‌های منبع‌باز بزرگ هستند.
- **یکپارچه‌سازی تحلیل‌گر مبتنی بر مسیر:** تحلیل‌گر ایستای clang که در فصل دو بررسی شد، در فاز یک به کار گرفته نشد، زیرا در حالت تک‌فایلی به فرآیند کامپایل و پیکربندی پروژه وابسته است. با پشتیبانی از فایل compile_commands.json و اجرای تحلیل در سطح پروژه، می‌توان این ابزار را به‌عنوان یک منبع نامزد دیگر در کنار ابزارهای الگومحور قرار داد و اثر آن بر دقت و یادآوری را جداگانه سنجید.
- **تکامل عامل زبانی با ابزار پرس‌وجو:** یکی از چشم‌اندازهای قدرتمند، توسعه ساختار بازبینی تک‌مرحله‌ای فعلی، به یک عامل تعاملی و چندمرحله‌ای است که مجهز به ابزار پرس‌وجو (Query tool) باشد. اگر عامل بتواند به‌جای اتکای صرف به یک قطعه ثابت و ارسال‌شده از کد، خود شخصاً به‌صورت پویا زمینه لازم را از فایل‌ها و ساختار کلی پروژه استخراج و بررسی کند، بدون شک انتظار می‌رود نرخ خروجی‌های نامطمئن در سیستم به‌طور چشمگیری کاهش یابد.
- **سنجش زنده نقش‌های تفکیک‌شده:** در نهایت، اجرای زنده و واقعی معماری دوگانه «جستجوگر و ارزیاب» روی همان محک‌های آزمایشی پایتون و زبان‌های C و C++ و مقایسه نتایج حاصل از آن با مسیر سنتی و تک‌عامله، می‌تواند داده‌های معتبری تولید کند تا سهم تأثیرگذاری تفکیک نقش‌ها از سهم کیفیت شواهد ارائه‌شده کاملاً متمایز و جداگانه ارزیابی شود.

فهرست مراجع

- [١] cppcheck, "A tool for static C/C++ code analysis." [Online]. Available: <https://cppcheck.sourceforge.io/>
- [٢] flawfinder, "A free, open source C/C++ program analyzer." [Online]. Available: <https://sourceforge.net/projects/flawfinder/>
- [٣] The clang static analyzer, "A static analyzer for C/C++ built on top of the LLVM compiler infrastructure." [Online]. Available: <https://clang-analyzer.llvm.org/>
- [٤] National Institute of Standards and Technology, "Juliet C/C++ Test Suite," in Software Assurance and Vulnerability Metrics, SARD test suites. [Online]. Available: <https://samate.nist.gov/SARD/test-suites/112>
- [٥] MITRE, "CWE-122: Heap-based Buffer Overflow." [Online]. Available: <https://cwe.mitre.org/data/definitions/122.html>
- [٦] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?," in Proc. 35th Int. Conf. Software Engineering (ICSE), 2013, pp. 672-681.
- [٧] N. Ayewah and W. Pugh, "The Google FindBugs fixit," in Proc. 19th Int. Symp. Software Testing and Analysis (ISSTA), 2010.
- [٨] D. Evans and D. Larochelle, "Improving security using extensible lightweight static analysis," in IEEE Software, vol. 19, no. 1, 2002, pp. 42-51.
- [٩] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," in Advances in Neural Information Processing Systems 32 (NeurIPS), 2019, pp. 10197-10207.
- [١٠] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," in Proc. IEEE Symp. Security and Privacy (SP), 2022, pp. 754-768.

- [۱۱] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. Zhang, “Large language models for software engineering: Survey and open problems,” in Proc. IEEE/ACM Int. Conf. Software Engineering: Future of Software Engineering (ICSE-FoSE), 2023, pp. 31-53.
- [۱۲] Tree-sitter contributors, “Tree-sitter documentation: an incremental parsing system for programming tools.” [Online]. Available: <https://tree-sitter.github.io/tree-sitter/>
- [۱۳] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459-9474.
- [۱۴] E. M. Bender et al., “On the dangers of stochastic parrots: Can language models be too big?,” in Proc. ACM Fairness, Accountability, and Transparency, 2021, pp. 610-623.
- [۱۵] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, “Do users write more insecure code with AI assistants?,” in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2023, pp. 2785-2799.
- [۱۶] National Institute of Standards and Technology, “CVE-2021-3156 Detail,” National Vulnerability Database. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2021-3156>
- [۱۷] National Institute of Standards and Technology, “CVE-2017-7529 Detail,” National Vulnerability Database. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2017-7529>
- [۱۸] National Institute of Standards and Technology, “CVE-2022-22817 Detail,” National Vulnerability Database. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2022-22817>
- [۱۹] National Institute of Standards and Technology, “CVE-2025-27516 Detail,” National Vulnerability Database. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2025-27516>
- [۲۰] National Institute of Standards and Technology, “CVE-2017-18342 Detail,” National Vulnerability Database. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2017-18342>
- [۲۱] National Institute of Standards and Technology, “CVE-2021-25239 Detail,” National Vulnerability Database. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2021-25239>